

부록

- AI와 이미지 처리



학습목표

- 이 워크샵에서는 인공지능과 이미지 처리에 확인할 수 있다

눈높이 체크

- 객체 검출-YOLO를 알고 계시나요?

1. 컴퓨터 비전의 현황

1. 다양한 공개 데이터 셋 활용

0 0 0 0 0 0
1 1 1 1 1 1
2 2 2 2 2 2
3 3 3 3 3 3
4 4 4 4 4 4
5 5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7
8 8 8 8 8 8
9 9 9 9 9 9

❖ MNIST



❖ Pascal VOC



❖ Quick! Draw!



❖ CelebA

이미지 출처 : MNIST 데이터베이스, Xiongwei Wu et al Recent Advances in Deep Learning for Object Detection 2019 arXiv:1908.03673
Quick! Draw! 데이터베이스, CelebA 데이터베이스

1. 컴퓨터 비전의 현황

2. ImageNet(2009-2017)



AI 연구를 변화시킨 공개 데이터

1 왜 데이터가 필요한가?

- 좋은 알고리즘이 좋은 결과를 도출?
- 현실과 동떨어진 개발에 대한 회의
- 데이터의 부족은 AI 성능 문제로 봉착

2 ImageNet 데이터베이스의 구축

- 2만 카테고리, 1400만 이미지
- 1000 카테고리, 120만 이미지로 경연 개최



이미지 출처: ImageNet

Ye, Tengqi. 2018. "Visual Object Detection from Lifelogs using Visual Non-lifelog Data."

1. 컴퓨터 비전의 현황

3. 한국형 이미지넷 구축 사업

1 이미지

- 사물, 글자체, 의료, 얼굴 등 다양한 공개 데이터셋 제공



이미지 출처 : eihub.or.kr 홈페이지

2 영상

- 인도 보행 영상, 딥페이크 감지용 영상 등

향후 지속적 데이터 구축 확대 예정

1. 컴퓨터 비전의 현황

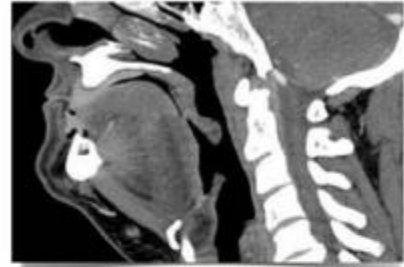
4. 컴퓨터 비전의 활용



자율주행



AR glass



의료영상

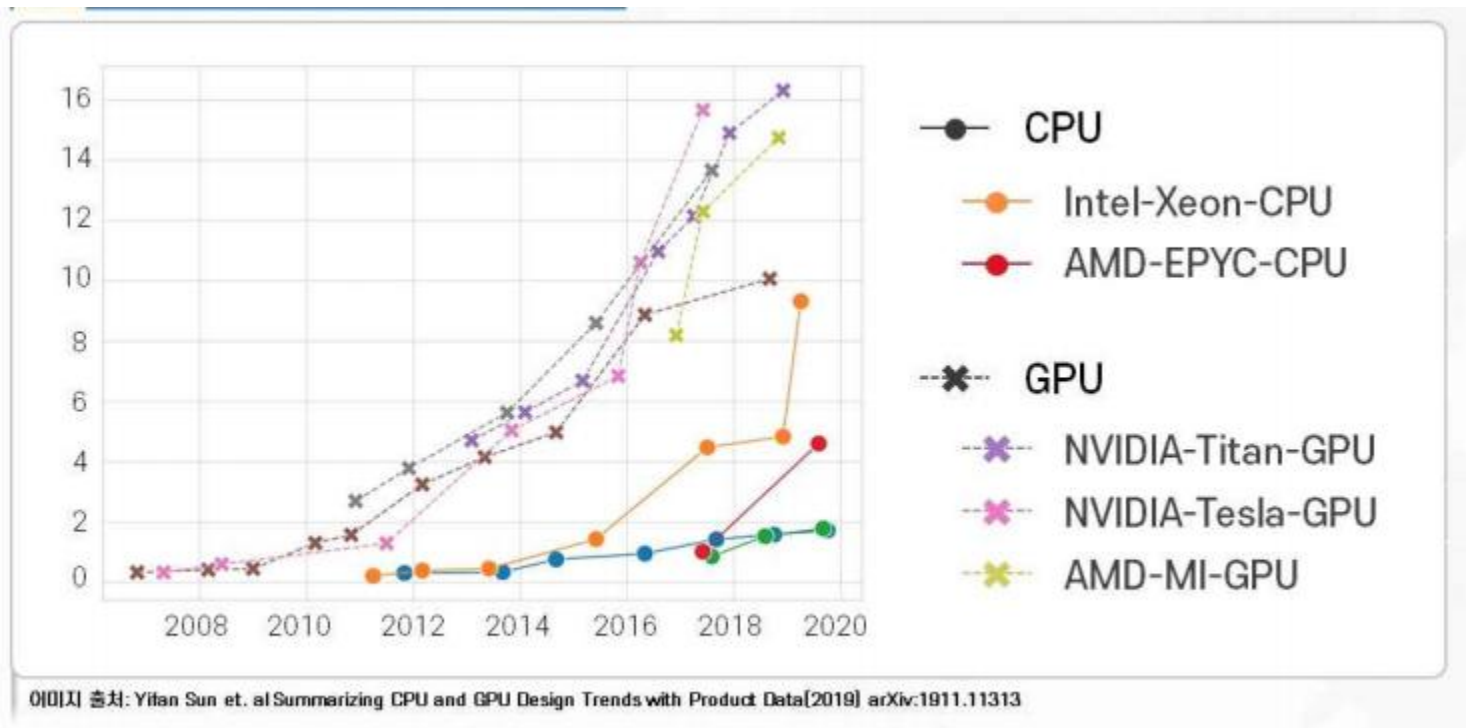
이미지 출처 : [https://www.gettyimagesbank.com/\(1263611579, 1053574558\)](https://www.gettyimagesbank.com/(1263611579, 1053574558)), en.wikipedia.org/wiki/X-ray

- ❖ 행동인식, 얼굴인식, 표정인식 등 지능형 카메라
- ❖ 인쇄체(서적), 필기체(우편봉투), 도로표지판, 간판 등의 문자 인식
- ❖ 드론영상, 농업영상, 공장 자동화 등



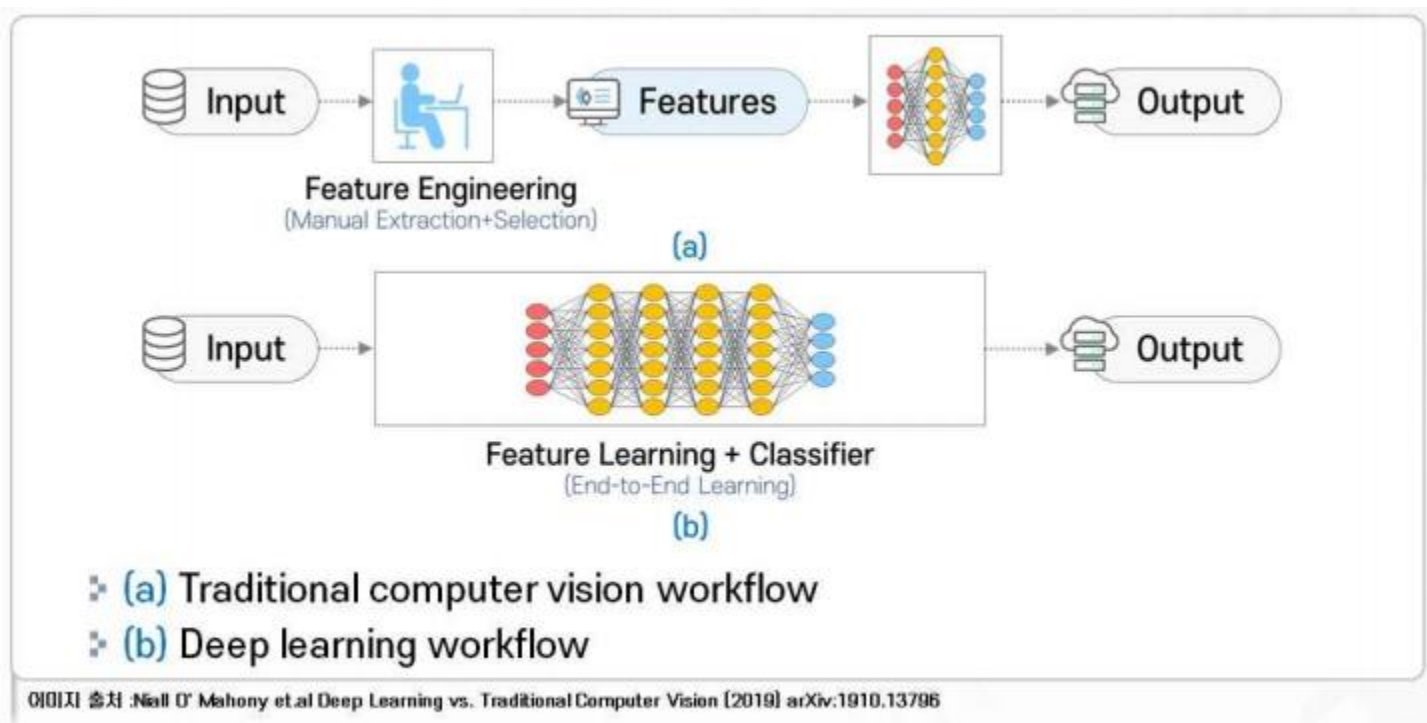
1. 컴퓨터 비전의 현황

5. GPU와 CPU



1. 컴퓨터 비전의 현황

6. 컴퓨터 비전에서의 고전적 방식과 딥러닝 차이





1. 컴퓨터 비전의 현황

7. 왜 컴퓨터 비전을 배워야 할까?

딥러닝 전문가 과정의 시작

컴퓨터 비전을 포함한
다양한 딥러닝 분야의
학습 및 연구의 기초가 됨

컴퓨터 비전을 이용한 응용 프로그램의 개발

- CCTV에 저장된 영상의 지능 처리
- 학교/사내/기관에 저장된 이미지의 재가공 및 정보 처리
- GAN 등을 이용한 새로운 이미지의 생성

딥러닝 프로젝트의 기획 혹은 관리

- 다양한 딥러닝 기술과 응용 과제를 결합하여 새로운 아이디어 도출
- 딥러닝 프로젝트 관리를 위한 기본 개념 습득

새로운 기술에 대한 탐구

성장하는 기술에 대한
이해를 통하여
사회적, 인문학적, 경제적
관점의 확대



2. 딥러닝 이전의 컴퓨터 비전

1. The Cat Experiment(1959)

1 마취된 고양이로 실험

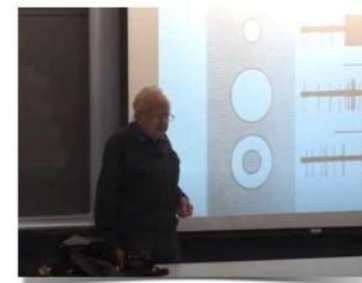
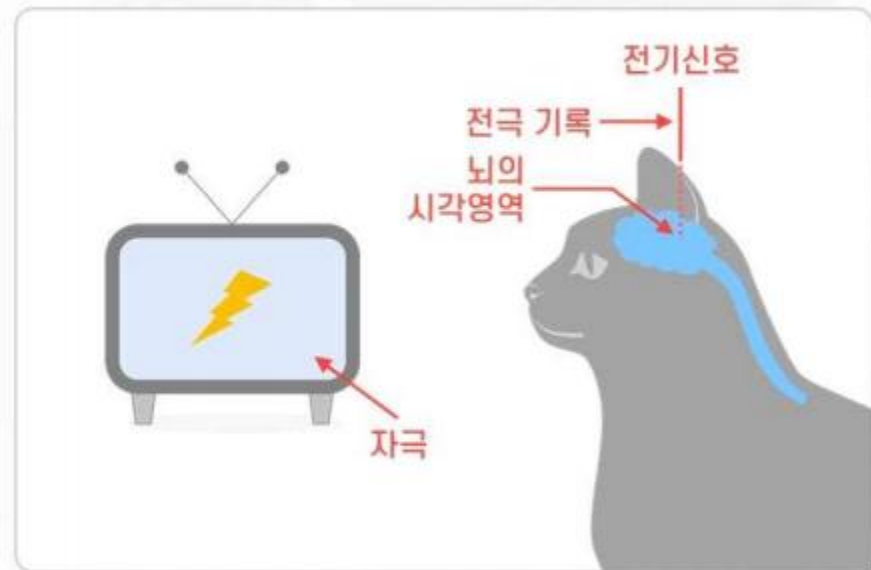
- 시각을 담당하는 뇌의 위치(시각 피질)에 전극 삽입

2 특정 시각 패턴을 보여줌

- 뇌의 활성화 영역 관찰
- 특정 패턴에 특정 영역이 활성화되는 것을 확인

3 두 종류의 활성화 영역

- Simple cells
- Complex cells

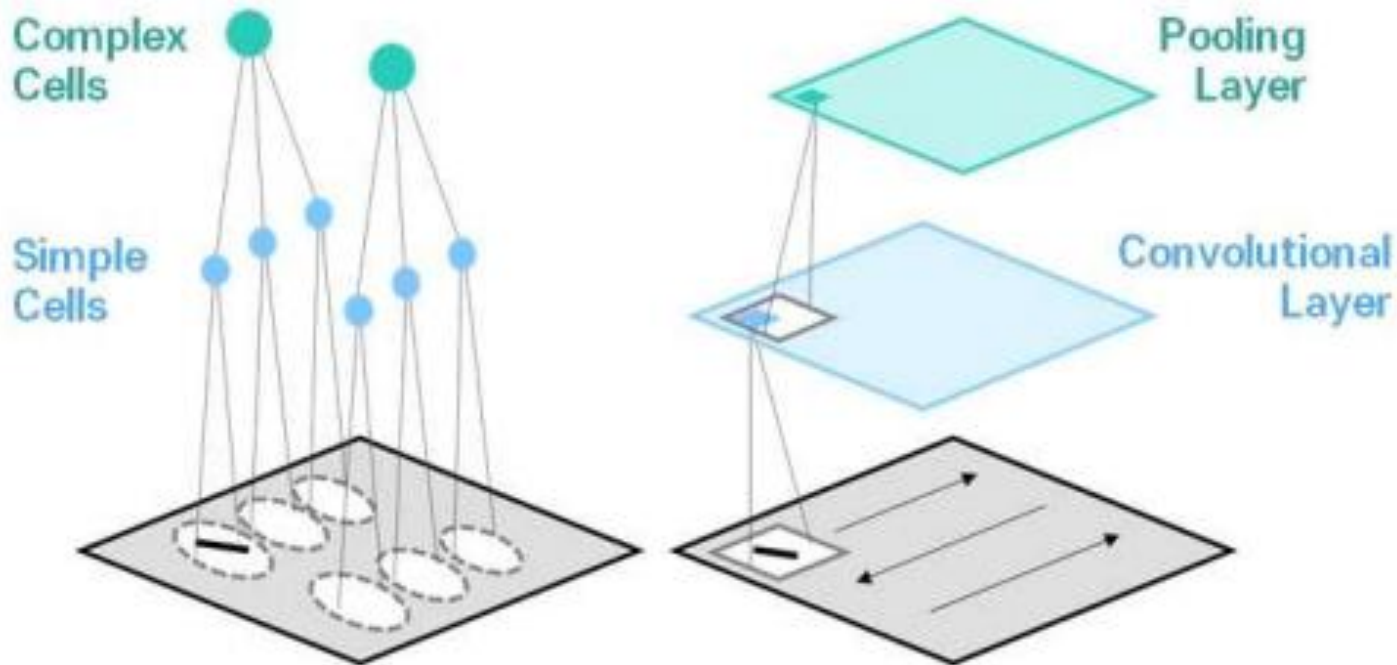


Wellesley 대학에서 고양이의 시각실험을 세미나 중인 Hubel



2. 딥러닝 이전의 컴퓨터 비전

1. The Cat Experiment(1959)

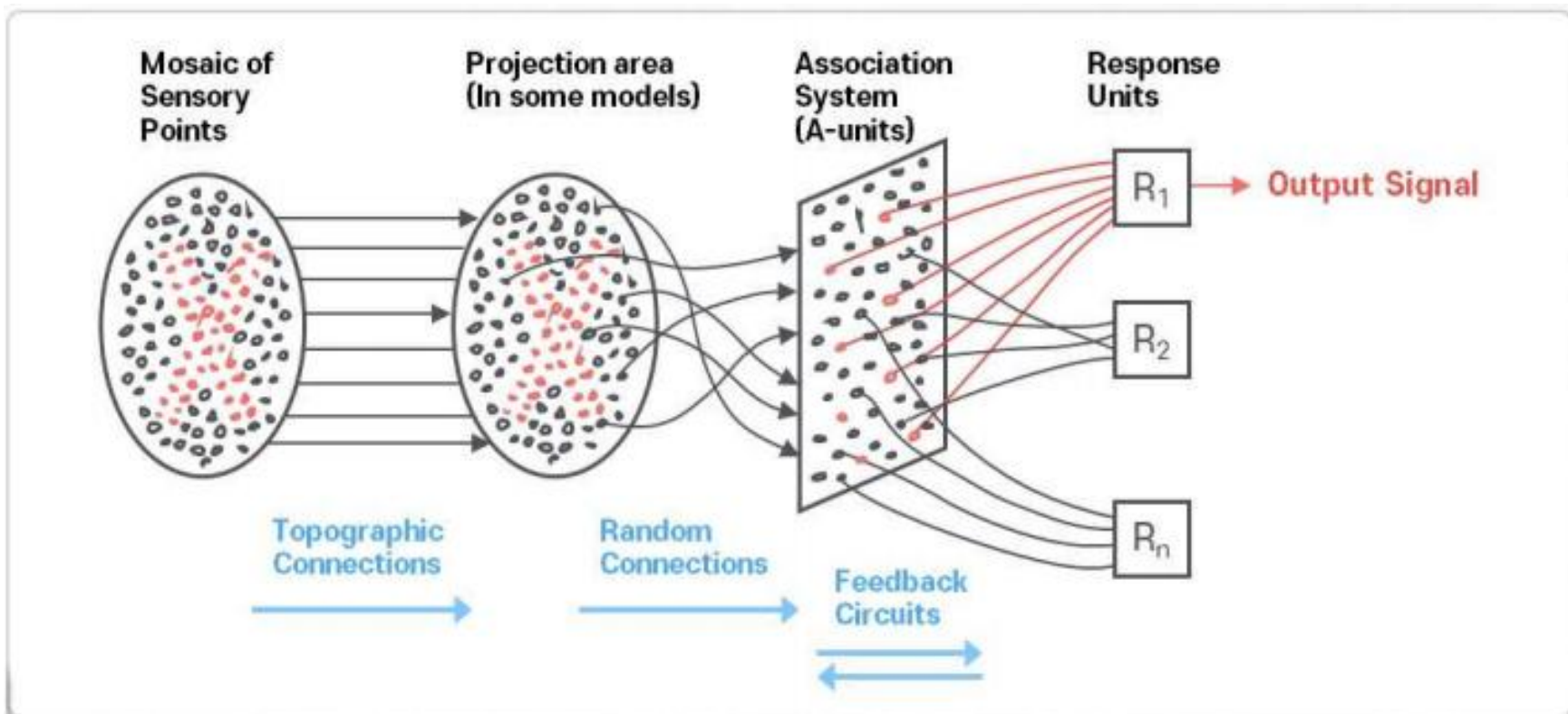


이미지 출처 : Grace W. Lindsay Convolutional Neural Networks as a Model of the Visual System: Past, Present, and Future(2020) arXiv:2001.07092



2. 딥러닝 이전의 컴퓨터 비전

2. Perceptron(1959)

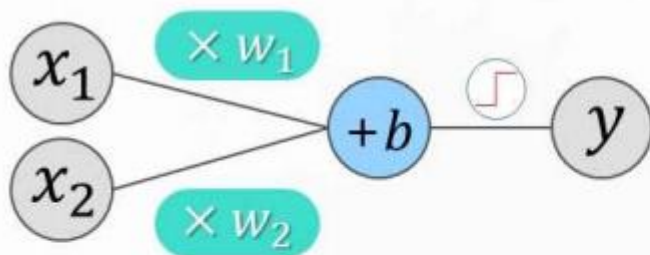


이미지 출처 :Rudolf Seising [2018] The Emergence of Fuzzy Sets in the Decade of the Perceptron—Lotti A. Zadeh's and Frank Rosenblatt's Research Work on Pattern Classification [<https://www.mdpi.com/2227-7390/6/7/110>]



2. 딥러닝 이전의 컴퓨터 비전

2. Perceptron(1959)



$$\begin{bmatrix} 1 \\ -1 \end{bmatrix} \longrightarrow 1$$

$$\begin{bmatrix} -1 \\ 1 \end{bmatrix} \longrightarrow -1$$

$$\hat{y} = \text{sign}(w_1x_1 + w_2x_2 + b)$$

$$w_1 \rightarrow w_1 + \eta(y - \hat{y})x_1$$

$$w_2 \rightarrow w_2 + \eta(y - \hat{y})x_2$$

$$b \rightarrow b + \eta(y - \hat{y})$$

$$(\eta = 0.1)$$



2. 딥러닝 이전의 컴퓨터 비전

2. Perceptron(1959)

- 1 학습 데이터 - 데이터의 수 총2개
 - $x = \begin{bmatrix} 1, -1 \\ -1, 1 \end{bmatrix}$ 입력 데이터
 - $y = \begin{bmatrix} 1, -1 \end{bmatrix}$ 출력 데이터 (Ground truth 참값)
- 2 추론 값 - 입력데이터가 주어질 때 신경망이 예측하는 값 (혹은 실행 값)
 - $\hat{y} = \text{sign}(w_1x_1 + w_2x_2 + b)$ 추론 값(inferred) 혹은 예측 값(predicted)
- 3 parameters
 - w : **weights**, b : **bios**
- 4 활성화 함수(Activation function)
 - $\text{sign}()$
- 5 Learning rate
 - η 보통 작은 값 (예: 0.1, 0.01, 0.0001 등)
- 6 batch size
 - parameter를 업데이트하는 테스트 데이터 개수 (여기서는 1)
- 7 epoch
 - 반복해서 학습하는 수 (parameter가 더이상 변하지 않을 때까지)



2. 딥러닝 이전의 컴퓨터 비전

2. Perceptron(1959)

8

손실함수 ^{맞보기!!}

- Ground truth와 prediction간의 차이를 측정하는 함수

$$L = (\hat{y} - y)^2 / 2$$

(신경망 학습 시간에 상세한 원리 설명 예정)

9

경사하강법(gradient descent)을 이용한 optimizer

- 손실함수를 미분하고 그 반대방향(마이너스)으로 학습

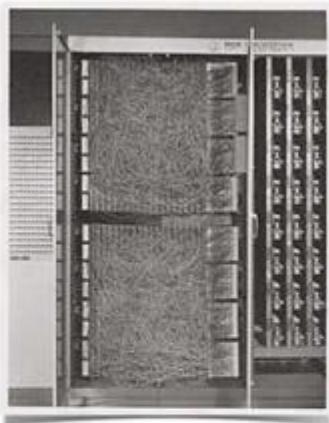
$$-\frac{dL}{dw_1} = -(\hat{y} - y)x_1$$

(신경망 학습 시간에 상세한 원리 설명 예정)



2. 딥러닝 이전의 컴퓨터 비전

2. Perceptron(1959)



❖ Mark I perceptron

20×20 화소 카메라 (400 픽셀)
Hard wired



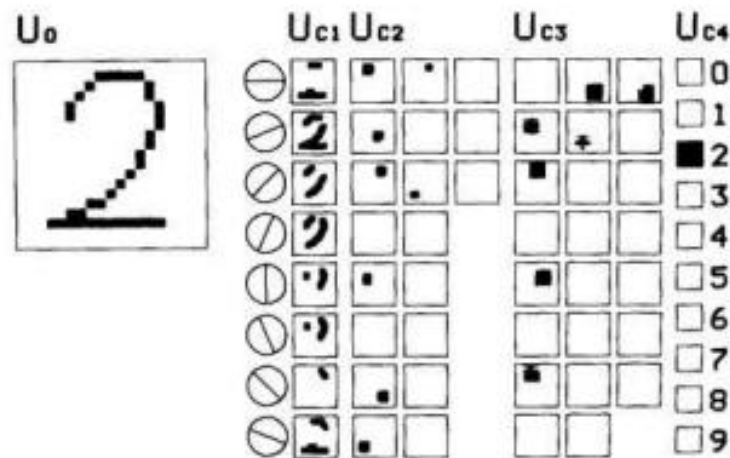
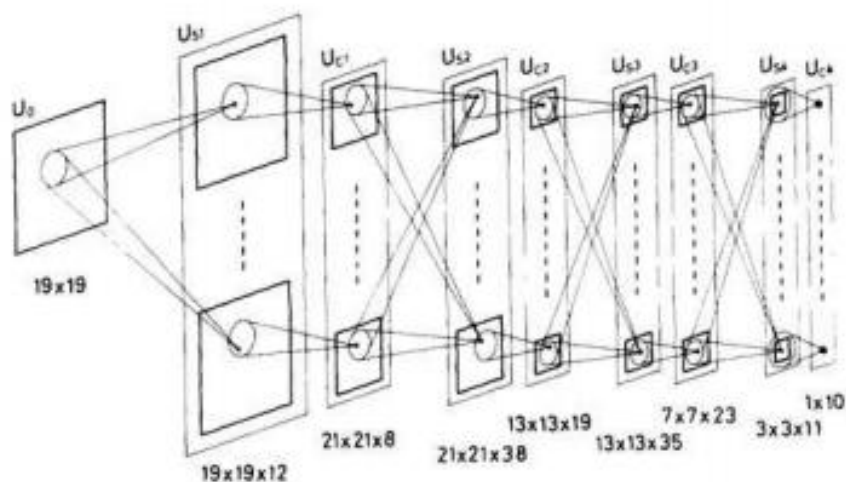
❖ Rosenblatt

“컴퓨터는 언젠가 걷고, 말하고, 보고
듣고, 재생산하며 자의식을 갖게 될 것”



2. 딥러닝 이전의 컴퓨터 비전

3. Neocognitron(1979)



❖ Hierarchical Network Structure of Neocognitron

이미지 출처 :Kunihiko Fukushima Neocognitron: A hierarchical neural network capable of visual pattern recognition(1988)



2. 딥러닝 이전의 컴퓨터 비전

4. Backpropagation(1986)

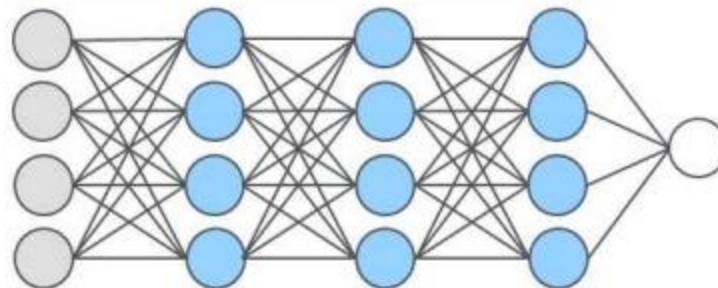
손실함수의 최소화 (경사하강법)

미분의 chain-rule 이용

Rumelhart, Hinton (1986)

2018년 Turing상 수상

Forward propagation 실행(추론) 시

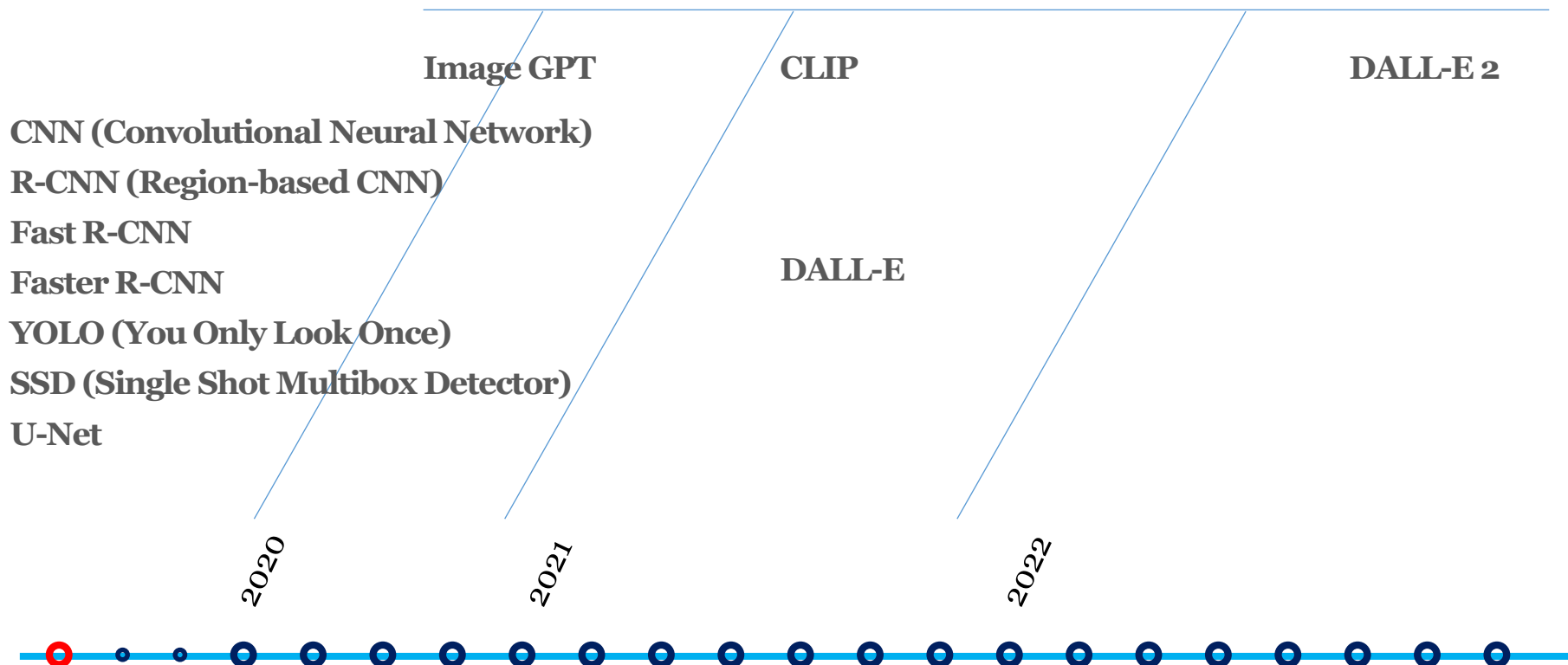


Backpropagation 학습 시



3. 딥러닝 모델의 영상 처리 적용

영상 처리 적용 딥러닝 모델의 역사





4. YOLO

YOLOv8 이란

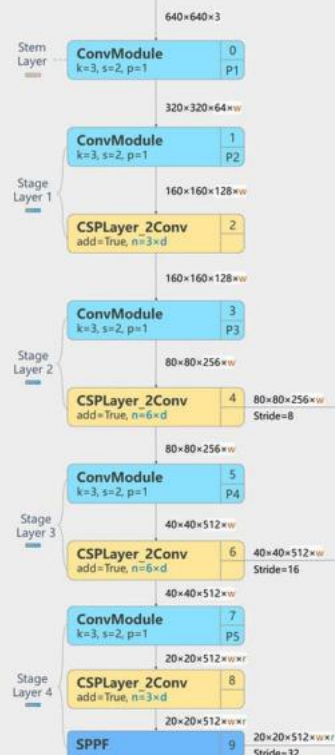
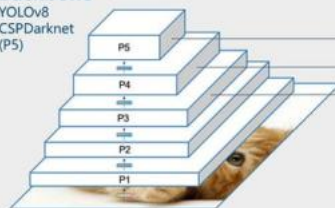
- YOLO 모델은 단일 단계의 객체 탐지기입니다. YOLO 모델에서 이미지 프레임은 백본(backbone)을 통해 피처화 됩니다.
- 이러한 피처는 넥에서 결합되고 혼합되며, 그런 다음 네트워크의 헤드로 전달됩니다. YOLO는 객체 주위에 경계 상자를 그릴 위치와 클래스를 예측합니다.
- 백본(Backbone)은 객체 탐지 모델에서 이미지에서 피처를 추출하는 데 사용되는 네트워크의 일부분입니다. 일반적으로, 백본은 컨볼루션(Convolution) 레이어와 풀링(Pooling) 레이어로 구성된 심층 신경망(Convolutional Neural Network, CNN)입니다.
- 백본의 역할은 입력 이미지에서 고수준의 추상적인 피처를 추출하여 객체의 특징을 표현하는 것입니다. 이 추출된 피처는 후속 단계에서 객체의 위치와 클래스를 예측하기 위해 사용됩니다. 백본은 일반적으로 이미지의 공간적인 정보를 보존하면서 이미지를 다운샘플링하여 피처의 크기를 줄이는 역할을 합니다. 이는 계산 효율성과 모델의 성능을 향상시키는 데 도움이 됩니다.
- YOLO의 경우, 다양한 버전에서 다양한 백본 아키텍처가 사용되며, 이는 모델의 정확도와 속도에 영향을 미칠 수 있습니다. 주로 ResNet, DarkNet, MobileNet 등의 백본 아키텍처가 YOLO 모델에서 많이 사용됩니다.

4. YOLO

YOLOv8 이란

YOLOv8

Backbone
YOLOv8
CSPDarknet
(P5)

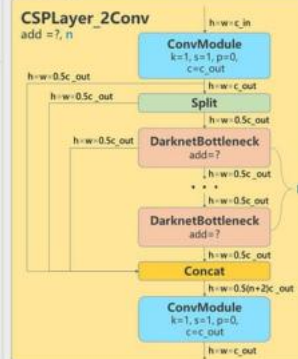


Backbone

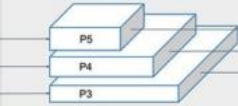
Neck
YOLOv8PAFPN



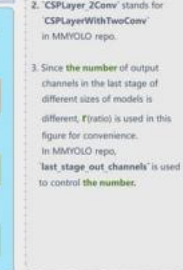
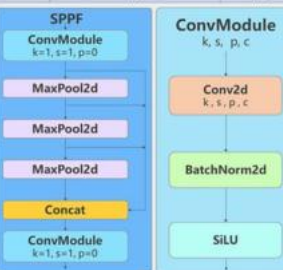
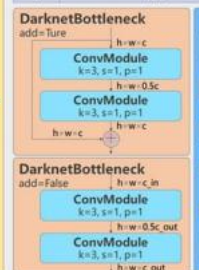
Details



Head
YOLOv8HeadModule



model	d (deepen factor)	w (widen factor)	r (ratio)
n	0.33	0.25	2.0
s	0.33	0.50	2.0
m	0.67	0.75	1.5
l	1.00	1.00	1.0
x	1.00	1.25	1.0



Note:

1. The numbers on the connecting line stand for height*width*channel
2. CSPLayer_2Conv stands for CSPLayerWithTwoConv in MMYOLO repo.
3. Since the number of output channels in the last stage of different sizes of models is different, F(ratio) is used in this figure for convenience. In MMYOLO repo, 'last_stage_out_channels' is used to control the number.

Neck

Head

YOLOv8 이란

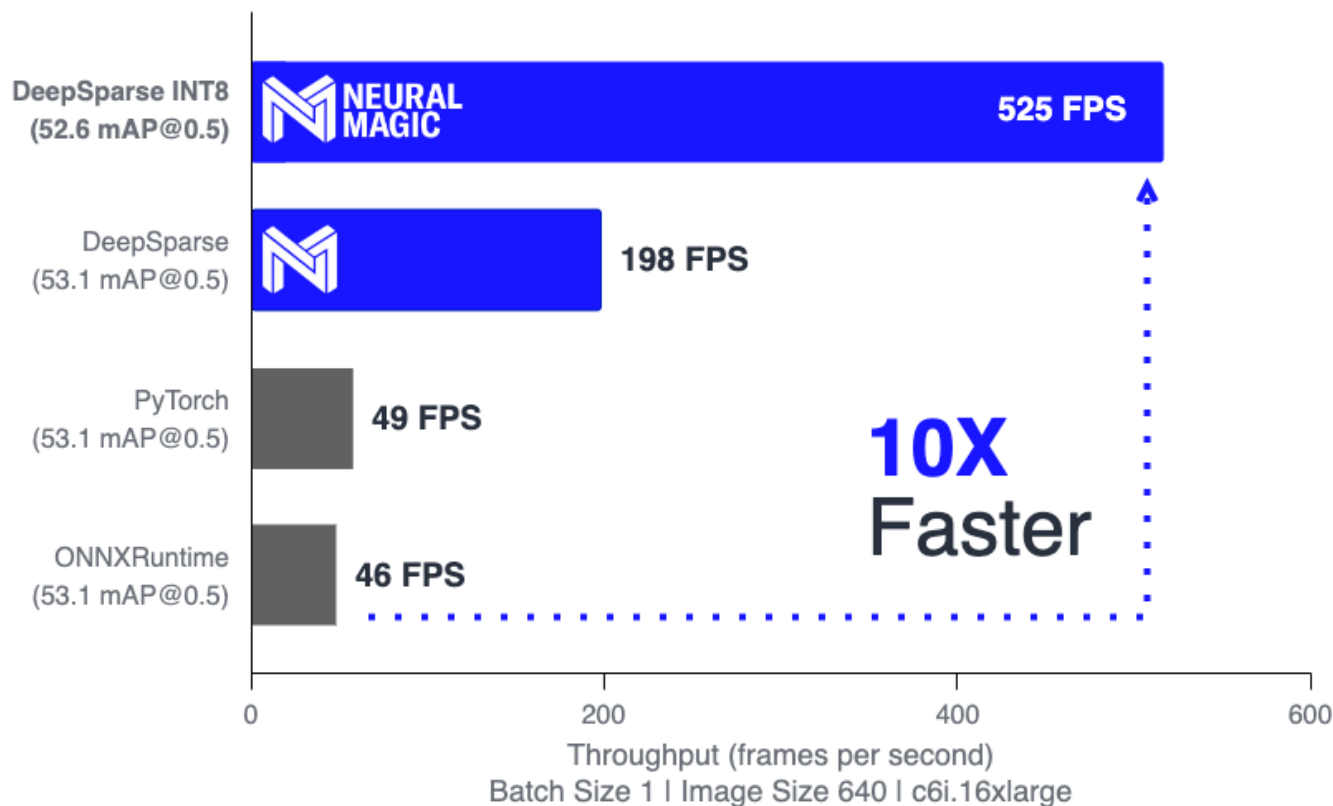
- YOLOv3와 YOLOv5 모델의 저자인 Ultralytics에서 개발되었으며, YOLOv8는 앵커 없는 디자인으로 객체 검출을 더욱 발전시켰습니다. 하지만 최신 정확도에만 중점을 두는 것은 아닙니다. YOLOv8는 실제 세계에서의 배포를 고려하여 속도, 지연 시간 및 가격 대비 성능에 초점을 맞췄습니다.
- YOLO의 최신 버전 및 DeepSparse를 사용하여 YOLOv8s(작은 버전)에서 209 FPS, YOLOv8n(나노 버전)에서 525 FPS를 달성하며, PyTorch 및 ONNX Runtime에 비해 10배의 가속을 얻을 수 있습니다!



4. YOLO

YOLOv8 이란

YOLOv8n FPS with DeepSparse





4. YOLO

YOLOv8를 사용하는 이유

- YOLOv8은 높은 정확도를 가지고 있으며, COCO 및 Roboflow 100에서 측정된 결과가 이를 입증하고 있습니다.
- 많은 개발자 편의 기능을 제공합니다. 사용하기 쉬운 CLI(Command Line Interface)와 잘 구성된 Python 패키지 등의 기능을 제공합니다.



4. YOLO

적용 가이드

- 적용 가이드는 아래 링크에 자세히 나와있습니다.
- <https://docs.ultralytics.com/>
- <https://dev.to/andreygermanov/how-to-create-yolov8-based-object-detection-web-service-using-python-julia-nodejs-javascript-go-and-rust-4o8e>
- <https://github.com/sjinzhang/awesome-yolo-object-detection>

Where to Start

- **Install** `ultralytics` with pip and get up and running in minutes

 **Get Started**

- **Predict** new images and videos with YOLOv8

 **Predict on Images**

- **Train** a new YOLOv8 model on your own custom dataset

 **Train a Model**

- **Explore** YOLOv8 tasks like segment, classify, pose and track

 **Explore Tasks**

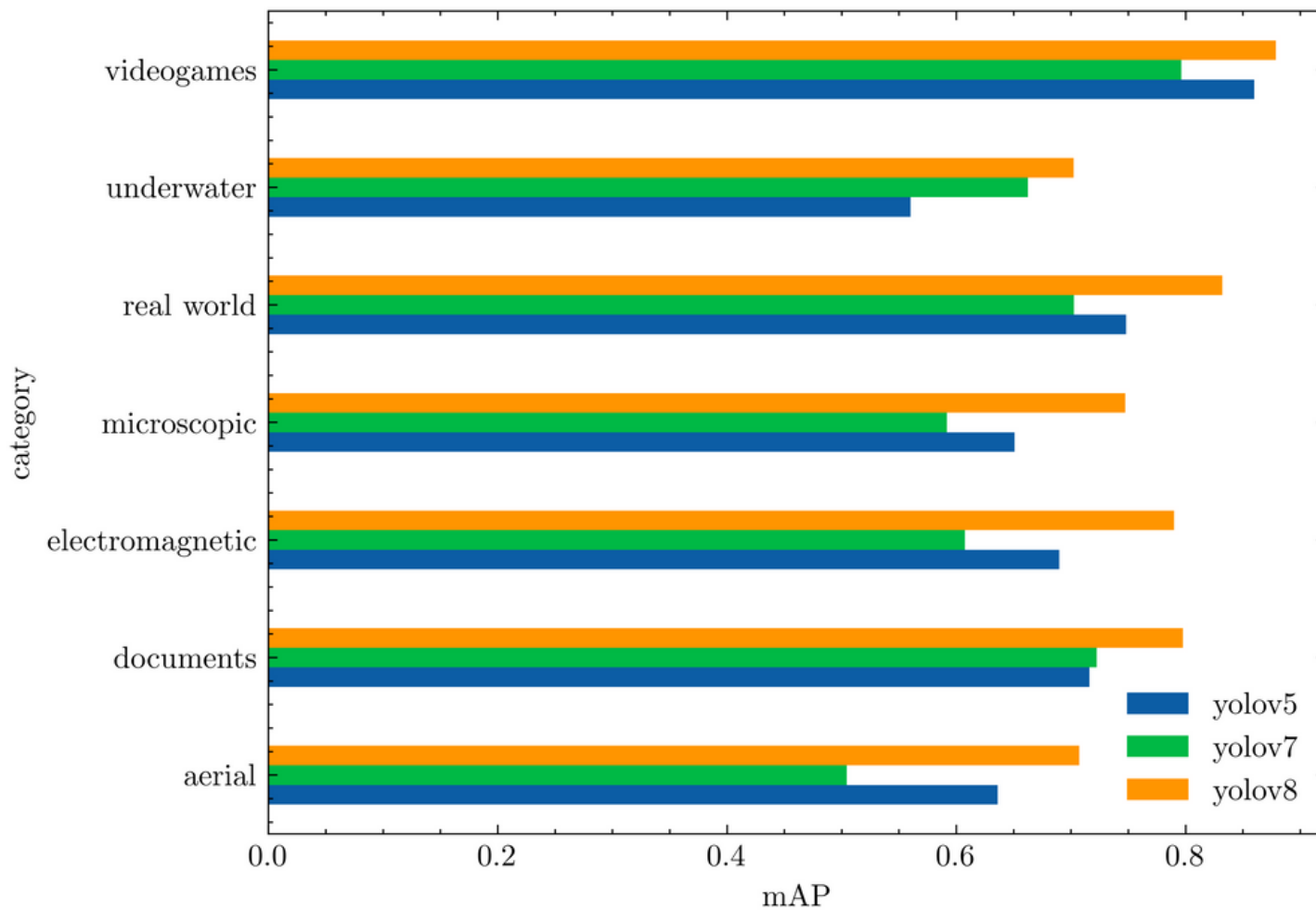
어디에 적용 할 수 있을까?

- 객체 추적은 시간에 따라 프레임 시퀀스에서 특정 객체 또는 여러 객체를 식별하는 과정입니다. 이는 각 프레임에서 객체의 위치를 파악하고 프레임 간 이동을 추적하는 것을 포함합니다. 객체 추적은 감시, 로봇공학, 자율 주행, 스포츠 분석 등 다양한 분야에서 다양한 응용 분야를 가지고 있습니다.
- 추적 알고리즘은 색상 히스토그램, 움직임 분석, 딥 러닝 등 다양한 기술을 사용하여 비디오 또는 이미지 시퀀스에서 객체를 찾고 따라가는 것입니다. 객체 감지와 추적이 다양한 산업에서 증가함에 따라 정확하고 실시간 추적 알고리즘의 필요성은 이전보다 더욱 중요해졌습니다.
- 객체를 정확하고 신속하게 추적하는 능력은 보안 감시와 같은 응용 분야에서 매우 중요합니다. 이를 통해 잠재적인 위협을 식별하고 수상한 행동을 탐지하는 데 도움을 줄 수 있습니다. 스포츠 분석 분야에서는 객체 추적을 통해 경기 중 선수들의 움직임을 분석하여 선수의 성과와 게임 전략에 대한 유용한 통찰력을 제공할 수 있습니다.



4. YOLO

어디에 적용 할 수 있을까?





4. YOLO

YOLO Licenses

● AGPL-3.0 License

- AGPL-3.0 라이선스는 상업적 이용을 허용합니다. 상업적으로 소프트웨어를 이용하고 수정하여 사용할 수 있습니다. 하지만 AGPL-3.0 라이선스의 조건을 준수해야 합니다.
- 이는 소스 코드의 공개, 라이선스 조건의 준수, 웹 애플리케이션의 경우 사용자에게 소스 코드 접근을 허용하는 등의 규정을 포함합니다.
- 따라서, AGPL-3.0 라이선스를 사용하는 소프트웨어를 상업적으로 이용하려면 해당 라이선스의 조건을 충족시켜야 합니다.
- 소스 코드의 공개와 라이선스 조건의 준수는 AGPL-3.0 라이선스의 핵심 요소이며, 이를 준수하지 않는다면 라이선스 위반으로 간주될 수 있습니다.
- 따라서, AGPL-3.0 라이선스를 사용하는 소프트웨어를 상업적으로 이용하고자 할 경우, 해당 라이선스의 조건을 자세히 검토하고 준수해야 합니다.



4. YOLO

YOLO Licenses

저자	발표시점	단체	출간처	특징	정확도(mAP)	속도 (fps)	논문/github	코드 베이스	라이선스	
YOLOv1	Joseph Redmon, et al.	2016년 5월	University of Washington	CVPR 2016	Single neural network approach to object detection as a regression problem	63.4% (VOC 2007)	45 fps (Titan X)	paper github	Darknet	"YOLO" license (All-permissive)
YOLOv2	Joseph Redmon, Ali Farhadi	2016년 12월	University of Washington	CVPR 2017	Anchor box, Batch normalization	78.6% (VOC 2007)	67 fps (Titan X)	paper github	Darknet	"YOLO" license
YOLOv3	Joseph Redmon, Ali Farhadi	2018년 4월	University of Washington	arXiv	Feature Pyramid Network (FPN), Multi-Scale Prediction	33.0% (COCO)	35 fps (Titan X)	paper github	Darknet	"YOLO" license
YOLOv4	Alexey Bochkovskiy, et al.	2020년 4월	(2,3저자) National Taiwan University	arXiv (YOLOv4) CVPR 2021 (Scaled-YOLOv4)	Bag of Freebies (BoF), Bag of Specials (BoS)	43.5% (COCO)	65 fps (V100)	paper(scaled) github (darknet) github(pytorch)	Darknet	"YOLO" license

4. YOLO

YOLO Licenses

저자	발표시점	단체	출간처	특징	정확도(mAP) 속도 (fps)	논문/github	코드 베이스	라이선스	
YOLOv5	Glenn Jocher	2020년 6월	Ultralytics	-	PyTorch Implementation, CSP backbone, Bi-directional Concatenation	93 fps (RTX4090) 50%(COCO)	github	PyTorch	AGPL 3.0
YOLOv6	Chuyi Li, et al.	2022년 6월	Meituan	arXiv	n(BiC) module, Anchor-aided training(AAT) strategy	52.8% (COCO) 116 fps (T4)	paper(v1.0) paper(v3.0) github	PyTorch	GPL 3.0
YOLOv7	Chien-Yao Wang*, et al. (*v4 논문의 2 저자)	2022년 7월	Academia Sinica	CVPR 2023	Extended-ELAN, Model Reparameterization	55.9% (COCO) 56 fps (V100)	paper github	YOLOv5 (PyTorch)	GPL 3.0
YOLOv8	Glenn Jocher, et al.	2023년 1월	Ultralytics	-	Anchor-free, Multi-task application	53.9% (COCO) 78 fps (T4)	website	PyTorch	AGPL 3.0



4. YOLO

YOLO Licenses

저자	발표시점	단체	출간처	특징	정확도(mAP)	속도 (fps)	논문/github	코드 베이스	라이선스	
YOLOv9	Chien-Yao Wang, et al.	2024년 2월	Academia Sinica	arXiv	Programmable Gradient Information (PGI), Generalized ELAN NMS-free training, spatial-channel separation, downsampling	55.6% (COCO)	73 fps (T4)	paper github	YOLOv5	(PyTorch)
YOLOv10	Ao Wang, et al.	2024년 5월	Tsinghua University	arXiv / NeurIPS 2024 poster		54.4% (COCO)	94 fps(T4)	paper github	Ultralytics(YOLOv8/PyTorch)	AGPL 3.0



4. YOLO

YOLO Licenses

- YOLOv8의 API는 사용성과 확장성이 v5보다 훨씬 개선되어, 다양한 시나리오에 활용되고 있다. 이후, 등장한 Ultralytics 프레임워크 기반으로 개발된 v9, v10 모델 구현체들도 같은 라이선스를 따르고 있음.



4. YOLO

YOLOv8 설치

```
c:\DEV>cd envs
```

```
c:\DEV\envs> python -m venv py3_10_yolov8
```

```
c:\DEV\envs> cd py3_10_yolov8
```

```
c:\DEV\envs\py3_10_yolov8>Scripts\activate
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>pip install ultralytics
```

```
Collecting ultralytics
```

```
  Downloading ultralytics-8.2.41-py3-none-any.whl (792 kB)
```

```
----- 793.0/793.0 kB 16.7 MB/s eta 0:00:00
```

```
Collecting py-cpuinfo
```

```
  Downloading py_cpuinfo-9.0.0-py3-none-any.whl (22 kB)
```

```
Collecting ultralytics-thop>=2.0.0
```

```
...
```



4. YOLO

YOLOv8 설치

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>pip install ipykernel
```

```
Collecting ipykernel
```

```
Using cached ipykernel-6.29.4-py3-none-any.whl (117 kB)
```

```
Requirement already satisfied: psutil in c:\dev\envs\py3_10_yolov8\lib\site-packages (from ipykernel) (6.0.0)
```

```
Collecting ipython>=7.23.1
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>python.exe -m pip install --upgrade pip
```

```
Requirement already satisfied: pip in c:\dev\envs\py3_10_yolov8\lib\site-packages (23.0.1)
```

```
Collecting pip
```

```
Using cached pip-24.1-py3-none-any.whl (1.8 MB)
```

```
...23.0.1
```

```
Uninstalling pip-23.0.1:
```

```
Successfully uninstalled pip-23.0.1
```

```
Successfully installed pip-24.1
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>python -m ipykernel install --user --name  
py3_10_yolov8 --display-name "py3_10_yolov8"
```

```
Installed kernelspec py3_10_yolov8 in C:\Users\k8s\AppData\Roaming\jupyter\kernels\py3_10_yolov8
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>deactivate
```

```
c:\DEV\envs\py3_10_yolov8>cd ..
```

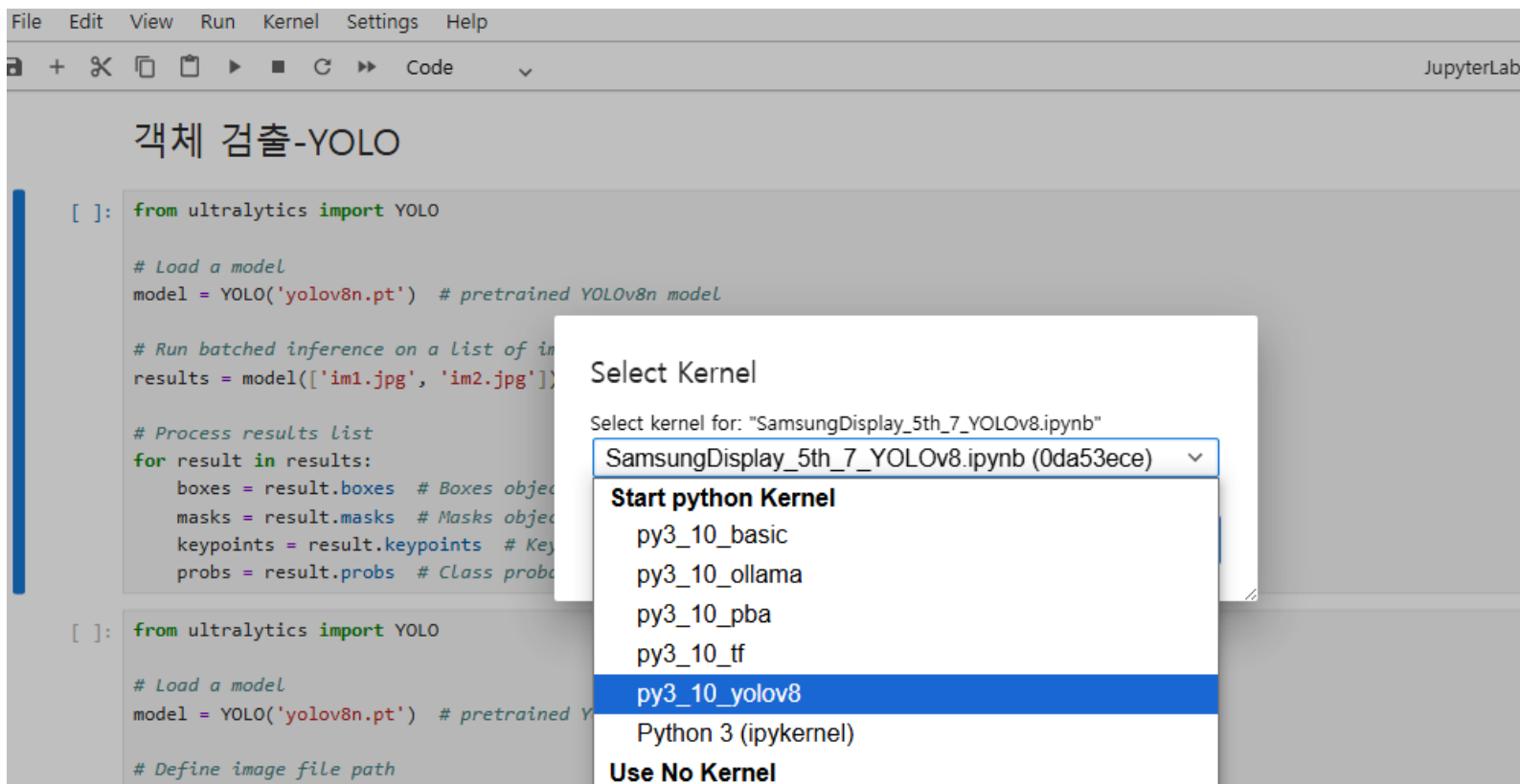
```
c:\DEV\envs>cd ..
```

```
c:\DEV>jupyter notebook
```

4. YOLO

YOLOv8 설치

c:\DEV>jupyter notebook



The screenshot shows a JupyterLab interface with a notebook titled "객체 검출-YOLO". The notebook contains two code cells. The first cell is partially obscured by a "Select Kernel" dialog box. The second cell is visible below the dialog. The dialog box shows the selected kernel as "SamsungDisplay_5th_7_YOLOv8.ipynb (0da53ece)" and lists several available kernels, with "py3_10_yolov8" highlighted.

```
[ ]: from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model

# Run batched inference on a list of images
results = model(['im1.jpg', 'im2.jpg'])

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding boxes predictions
    masks = result.masks # Masks object for segmentation masks predictions
    keypoints = result.keypoints # Keypoints object for pose estimation predictions
    probs = result.probs # Class probabilities predictions
```

```
[ ]: from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model

# Define image file path
```

Select Kernel

Select kernel for: "SamsungDisplay_5th_7_YOLOv8.ipynb"

SamsungDisplay_5th_7_YOLOv8.ipynb (0da53ece) ▾

Start python Kernel

- py3_10_basic
- py3_10_ollama
- py3_10_pba
- py3_10_tf
- py3_10_yolov8**
- Python 3 (ipykernel)

Use No Kernel



4. YOLO

공식 홈페이지에 안내하는 코드

```
from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model

# Run batched inference on a list of images
results = model(['im1.jpg', 'im2.jpg']) # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs
```

❖ 경로 문제로 에러남



4. YOLO

공식 홈페이지에 안내하는 코드

C:\aabb\serch\yolotest

C:\aabb\serch\yolotest





4. YOLO

공식 홈페이지에 안내하는 코드 수정

```
from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model

# Define image file path
image_folder = 'C:\\DEV\\KoreaTech\\YOLO\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2]) # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs
```

[illegible]

```
0: 256x640 15 persons, 1 car, 3 buss, 171.5ms
1: 256x640 15 persons, 1 car, 3 buss, 171.5ms
Speed: 2.5ms preprocess, 171.5ms inference, 4.6ms postprocess per image at shape (1, 3, 256, 640)
```

1. pretrained 모델 yolov8n.pt 파일을 불러오는데 여기서 이 용어는
2. “pretrained”는 미리 학습된 모델을 의미합니다. 딥 러닝 모델은 대량의 데이터셋에서 학습하여 일반적인 이미지 인식, 객체 감지, 자연어 처리 등의 작업을 수행할 수 있도록 학습됩니다. 하지만 이러한 모델을 처음부터 학습 시키는 데에는 많은 시간과 계산 리소스가 필요합니다.
3. 따라서, pretrained 모델은 이미 대규모 데이터셋에서 사전에 학습되어 일반적인 패턴과 특징을 학습한 모델을 의미합니다. 이러한 모델을 사용하면 처음부터 학습시키는 대신, 이전에 학습된 모델을 기반으로 새로운 작업에 대한 예측을 수행할 수 있습니다. pretrained 모델은 일반적으로 이미지넷(ImageNet)과 같은 대형 데이터셋에서 사전 학습된 모델을 의미합니다. 이 모델들은 이미지 분류, 객체 감지, 세그멘테이션 등 다양한 컴퓨터 비전 작업에 사용될 수 있습니다. pretrained 모델을 사용하면 소규모 데이터셋에서도 좋은 성능을 얻을 수 있고, 학습 시간과 비용을 절약할 수 있습니다.



4. YOLO

공식 홈페이지에 안내하는 코드 수정

참고로 이미지 입력 방법

Source	Argument	Type	Notes
image	'image.jpg'	str or Path	Single image file.
URL	'https://ultralytics.com/images/bus.jpg'	str	URL to an image.
screenshot	'screen'	str	Capture a screenshot.
PIL	Image.open('im.jpg')	PIL.Image	HWC format with RGB channels.
OpenCV	cv2.imread('im.jpg')	np.ndarray of uint8 (0-255)	HWC format with BGR channels.
numpy	np.zeros((640,1280,3))	np.ndarray of uint8 (0-255)	HWC format with BGR channels.
torch	torch.zeros(16,3,320,640)	torch.Tensor of float32 (0.0-1.0)	BCHW format with RGB channels.
CSV	'sources.csv'	str or Path	CSV file containing paths to images, videos, or directories.
video 	'video.mp4'	str or Path	Video file in formats like MP4, AVI, etc.
directory 	'path/'	str or Path	Path to a directory containing images or videos.
glob 	'path/*.jpg'	str	Glob pattern to match multiple files. Use the * character as a wildcard.
YouTube 	'https://youtu.be/Zgi9g1ksQHc'	str	URL to a YouTube video.
stream 	'rtsp://example.com/media.mp4'	str	URL for streaming protocols such as RTSP, RTMP, or an IP address.



4. YOLO

박스형상으로 확인

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model
model.predict(source="0", show=True, stream=True, classes=0) # [0, 3, 5] for multiple classes

# Define image file path
image_folder = 'C:\\DEV\\KoreaTech\\YOLO\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2]) # return a list of Results objects
model.predict(source="0", show=True, stream=True, classes=0)

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

# Display result with boxes
result.plot()
cv2.waitKey(0)
cv2.destroyAllWindows()
```



4. YOLO

박스형상 으로 확인

`for result in results:` 각 이미지에 대한 결과를 반복합니다.
`result.bboxes:` 바운딩 박스 객체를 가져옵니다.
`result.masks:` 세그멘테이션 마스크 객체를 가져옵니다.
`result.keypoints:` 포즈 추정 키포인트 객체를 가져옵니다.
`result.probs:` 분류 출력에 대한 클래스 확률을 가져옵니다.
`print(box):` 바운딩 박스 좌표를 출력합니다.
`result.plot():` 결과 이미지에 바운딩 박스를 그려서 표시합니다.
`cv2.waitKey(0), cv2.destroyAllWindows():` 결과 이미지를 보여준 후 키 입력을 기다리고 모든 창을 닫습니다.



4. YOLO

특정 사물만 인식

YOLOv8 특정 사물만 인식하는 방법은 아주 간단합니다.

`classes=0` 이 부분에 숫자가 인식하고자 하는 형상 번호 입니다.

```
model.predict(source="0", show=True, stream=True, classes=0) # [0, 3, 5] for multiple classes
```



4. YOLO

특정 사물만 인식

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('yolov8n.pt') # pretrained YOLOv8n model
model.predict(source="0", show=True, stream=True, classes=0) # [0, 3, 5] for multiple classes

# Define image file path
image_folder = 'C:\\DEV\\KoreaTech\\YOLO\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2]) # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



4. YOLO

코드 개선

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Load a pretrained YOLO model
model = YOLO("yolov8n.pt") # COCO dataset으로 pretrained된 모델을 불러옴

# Define image file path
image_path = "C:\\DEV\\KoreaTech\\YOLO\\im1.jpg"

# Use the model to perform object detection
results = model.predict(image_path)

# Print results for debugging
print(results[0])
print()
print(results[0].boxes) # 좌상단 좌표, 우하단 좌표, confidence score, class id

# Plot the result
res_plotted = results[0].plot() # 결과 이미지를 그리니다 (bounding box 등)

# Convert BGR (OpenCV format) to RGB (matplotlib format)
res_plotted_rgb = cv2.cvtColor(res_plotted, cv2.COLOR_BGR2RGB)

# Display the image using matplotlib
plt.figure(figsize=(12, 12))
plt.imshow(res_plotted_rgb)
plt.axis('off') # 축을 숨깁니다
plt.show()
```

4. YOLO

코드 개선

코드를 실행하면 이전에 차량과 사람을 모두 인식했던 결과와 다르게 사람만 인식한 결과를 확인할 수 있습니다.



4. YOLO

YOLOv8의 예측 결과를 그림 위에 "탐지한 객체 수"를 노란색 글씨로 표시

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

```
# 1. Load pretrained YOLO model
model = YOLO("yolov8n.pt")
```

```
# 2. Load image
image_path = "C:\\DEV\\KoreaTech\\YOLO\\im1.jpg"
```

```
# 3. Object detection
results = model.predict(image_path)
result = results[0]
boxes = result.boxes
```

```
# 4. Count detected objects
num_objects = len(boxes)
```

```
# 5. Draw results on image
res_plotted = result.plot() # OpenCV 이미지 (BGR)
text = f"Detected objects: {num_objects}"
```

```
# 6. Put yellow text on image (좌상단에 표시)
```

```
cv2.putText(
    res_plotted,          # 이미지
    text,                 # 출력할 텍스트
    (30, 50),             # 텍스트 위치 (x, y)
    cv2.FONT_HERSHEY_SIMPLEX, # 폰트
    1.3,                  # 폰트 크기 (13pt 정도에 해당)
    (0, 255, 255),        # 노란색 (BGR)
    2,                    # 두께
    cv2.LINE_AA           # 안티앨리어싱
)
```

```
# 7. Convert BGR to RGB for matplotlib
res_plotted_rgb = cv2.cvtColor(res_plotted, cv2.COLOR_BGR2RGB)
```

```
# 8. Show the image
plt.figure(figsize=(12, 12))
plt.imshow(res_plotted_rgb)
plt.axis('off')
plt.show()
```



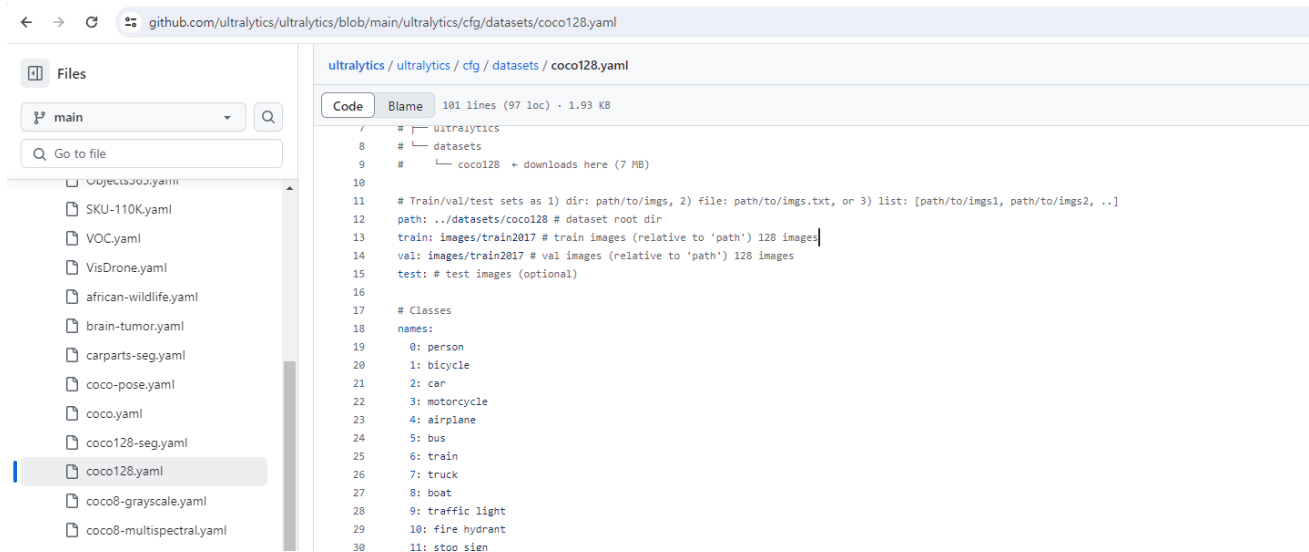
4. YOLO

선행된(pre-trained) YOLOv8 형상 클래스

YOLOv8 특정 사물만 인식하는 방법에 필요한 클래스(Class)는 아래의 80가지 클래스로 학습이 되어 있습니다.

<https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/datasets/coco128.yaml>

names:
0: person
1: bicycle
2: car
3: motorcycle
4: airplane
5: bus
6: train
7: truck
8: boat
9: traffic light
10: fire hydrant
11: stop sign
12: parking meter
13: bench
14: bird
15: cat
16: dog
17: horse
18: sheep
19: cow
20: elephant
21: bear
22: zebra
23: giraffe
24: backpack
25: umbrella
26: handbag
27: tie
28: suitcase
29: frisbee
30: skis
31: snowboard
32: sports ball
33: kite
34: baseball bat
35: baseball glove



```
7 # Ultralytics
8 # datasets
9 # coco128 ← downloads here (7 MB)
10
11 # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]
12 path: ../datasets/coco128 # dataset root dir
13 train: images/train2017 # train images (relative to 'path') 128 images
14 val: images/train2017 # val images (relative to 'path') 128 images
15 test: # test images (optional)
16
17 # Classes
18 names:
19 0: person
20 1: bicycle
21 2: car
22 3: motorcycle
23 4: airplane
24 5: bus
25 6: train
26 7: truck
27 8: boat
28 9: traffic light
29 10: fire hydrant
30 11: stop sign
```



4. YOLO

선행된(pre-trained) YOLOv8 형상 클래스

YOLOv8 특정 사물만 인식하는 방법에 필요한 클래스(Class)는 아래의 80가지 클래스로 학습이 되어 있습니다.

<https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/datasets/coco128.yaml>

36: skateboard
37: surfboard
38: tennis racket
39: bottle
40: wine glass
41: cup
42: fork
43: knife
44: spoon
45: bowl
46: banana
47: apple
48: sandwich
49: orange
50: broccoli
51: carrot
52: hot dog
53: pizza
54: donut
55: cake
56: chair
57: couch
58: potted plant
59: bed
60: dining table
61: toilet
62: tv
63: laptop
64: mouse
65: remote
66: keyboard
67: cell phone
68: microwave
69: oven
70: toaster
71: sink
72: refrigerator
73: book
74: clock
75: vase
76: scissors
77: teddy bear
78: hair drier
79: toothbrush

```
7 # ultralytics
8 # datasets
9 # coco128 ← downloads here (7 MB)
10
11 # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]
12 path: ../datasets/coco128 # dataset root dir
13 train: images/train2017 # train images (relative to 'path') 128 images
14 val: images/train2017 # val images (relative to 'path') 128 images
15 test: # test images (optional)
16
17 # Classes
18 names:
19 0: person
20 1: bicycle
21 2: car
22 3: motorcycle
23 4: airplane
24 5: bus
25 6: train
26 7: truck
27 8: boat
28 9: traffic light
29 10: fire hydrant
30 11: stoo sian
```



4. YOLO

커스텀 데이터 셋 개요

- YOLO를 통해 custom image를 학습시키는 경우 데이터셋이 필요하다.
- 자신이 원하는 데이터셋이 존재하지 않을 경우 직접 데이터셋을 만들어줘야 하는데, Image에서 Bounding Box를 지정하여 라벨링을 수행하고 Bounding Box 정보들을 좌표파일로 생성해야 한다.
- 이미지 라벨링 툴 labellmg를 이용해서 이미지 라벨링을 수행할 수 있다.



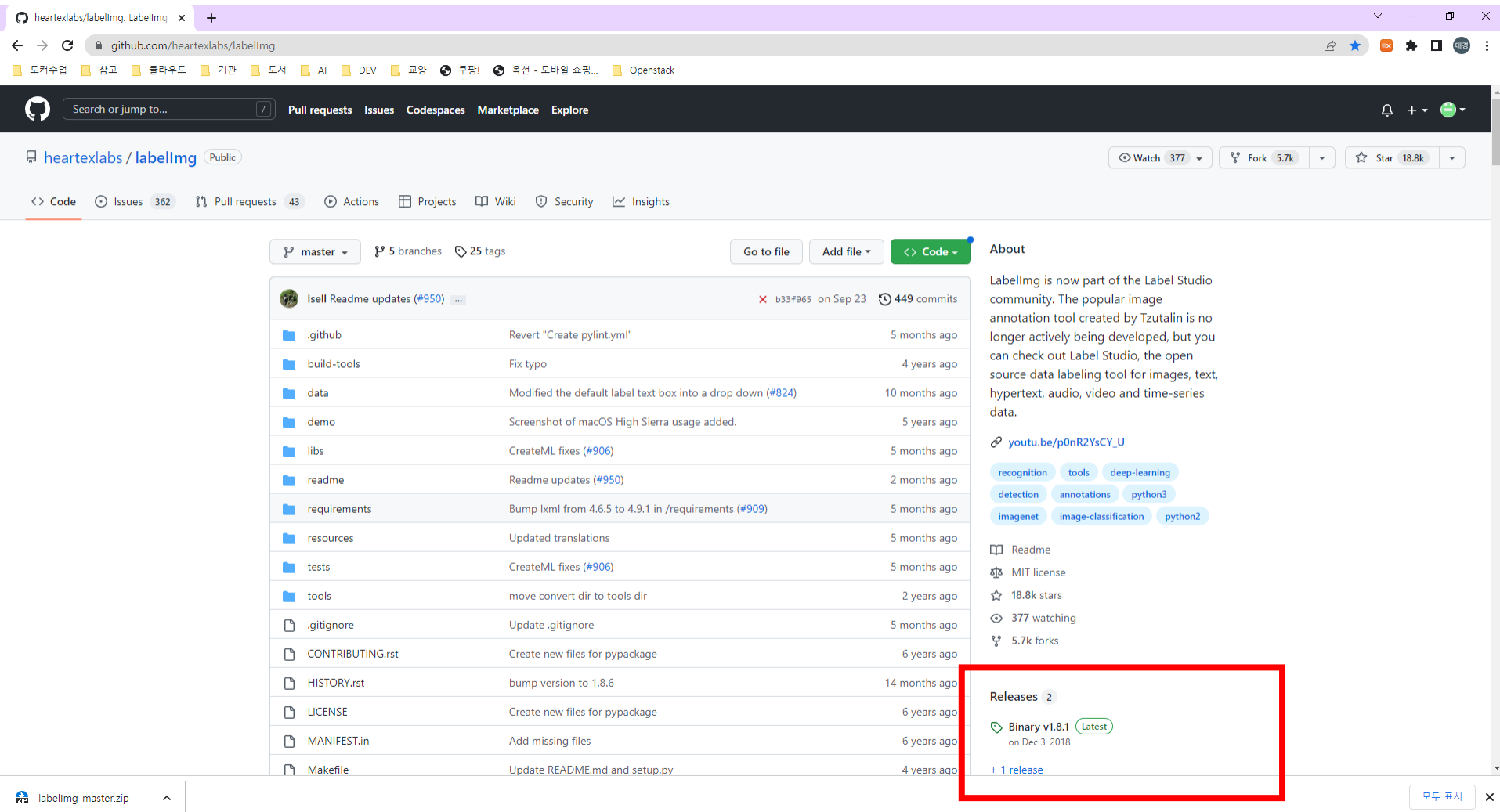
4. YOLO

설치 및 실행

- <https://github.com/heartexlabs/labelImg>
- <https://github.com/tzutalin/labelImg>
- 위 링크에 접속하면 다음과 같은 화면을 볼 수 있을 것이다.
- 그 중에서 Releases를 클릭해주자.

4. YOLO

설치 및 실행



heartexlabs/labelimg: Labelimg x +

github.com/heartexlabs/labelimg

도커수업 참고 클라우드 기관 도서 AI DEV 교양 루팡! 육선 - 모바일 소핑... Openstack

Search or jump to... Pull requests Issues Codespaces Marketplace Explore

heartexlabs/labelimg Public

Watch 377 Fork 5.7k Star 18.8k

<> Code Issues 362 Pull requests 43 Actions Projects Wiki Security Insights

master 5 branches 25 tags

Go to file Add file <> Code

Isell Readme updates (#950) b33f965 on Sep 23 449 commits

.github	Revert "Create pylint.yml"	5 months ago
build-tools	Fix typo	4 years ago
data	Modified the default label text box into a drop down (#824)	10 months ago
demo	Screenshot of macOS High Sierra usage added.	5 years ago
libs	CreateML fixes (#906)	5 months ago
readme	Readme updates (#950)	2 months ago
requirements	Bump lxml from 4.6.5 to 4.9.1 in /requirements (#909)	5 months ago
resources	Updated translations	5 months ago
tests	CreateML fixes (#906)	5 months ago
tools	move convert dir to tools dir	2 years ago
.gitignore	Update .gitignore	5 months ago
CONTRIBUTING.rst	Create new files for pypackage	6 years ago
HISTORY.rst	bump version to 1.8.6	14 months ago
LICENSE	Create new files for pypackage	6 years ago
MANIFEST.in	Add missing files	6 years ago
Makefile	Update README.md and setup.py	4 years ago

About

Labelimg is now part of the Label Studio community. The popular image annotation tool created by Tzutalin is no longer actively being developed, but you can check out Label Studio, the open source data labeling tool for images, text, hypertext, audio, video and time-series data.

youtu.be/p0nR2YsCY_U

recognition tools deep-learning detection annotations python3 imagenet image-classification python2

Readme MIT license 18.8k stars 377 watching 5.7k forks

Releases 2

Binary v1.8.1 Latest on Dec 3, 2018

+ 1 release

labelimg-master.zip

4. YOLO

설치 및 실행

Release Binary v1.8.1 · heartexlabs

github.com/heartexlabs/labelimg/releases/tag/v1.8.1

도커수업 참고 클라우드 기관 도서 AI DEV 교양 쿠팡! 육선 - 모바일 쇼핑... Openstack

Search or jump to... Pull requests Issues Codespaces Marketplace Explore

heartexlabs / labelimg Public

Watch 377 Fork 5.7k Star 18.8k

<> Code Issues 362 Pull requests 43 Actions Projects Wiki Security Insights

Releases / v1.8.1

Binary v1.8.1 Latest

tzutalin released this Dec 3, 2018 · 142 commits to master since this release v1.8.1 3f14cbf

windows_v1.8.1.zip

Assets 2

Source code (zip)	Dec 3, 2018
Source code (tar.gz)	Dec 3, 2018

49 3 52 people reacted

© 2022 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

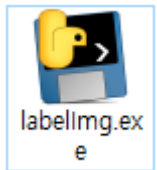
labelimg-master.zip

모두 표시

4. YOLO

설치 및 실행

- 여러가지 버전이 표시될텐데, Windows 환경이므로 Windows_v1.8.1.zip을 다운로드 해준 뒤, 압축을 해제한다.



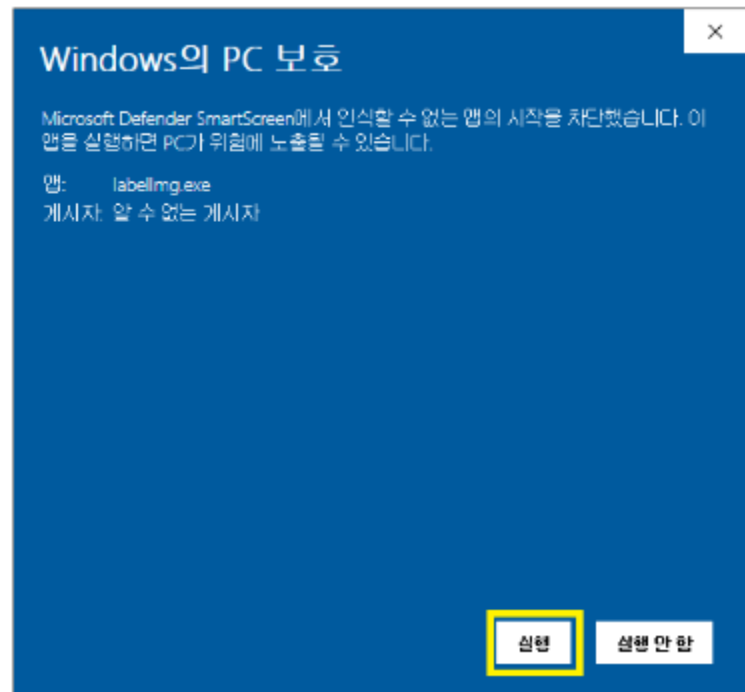
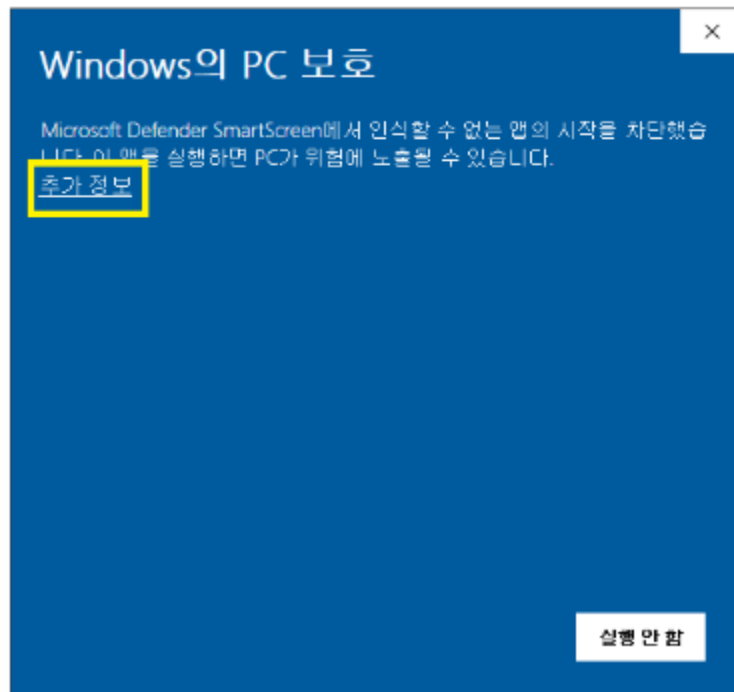
PC > 로컬 디스크 (C:) > DEV > windows_v1.8.1 >		
이름	수정한 날짜	크기
data	2018-12-03 오후 10:22	10.5 MB
labellmg.exe	2018-12-03 오후 10:22	10.5 MB

- labellmg.exe 파일을 실행시켜보자.

4. YOLO

설치 및 실행

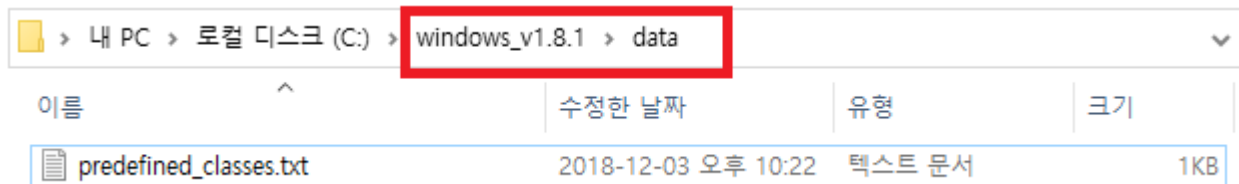
- 추가 정보 클릭 후 실행하면 된다.



4. YOLO

Class 정의

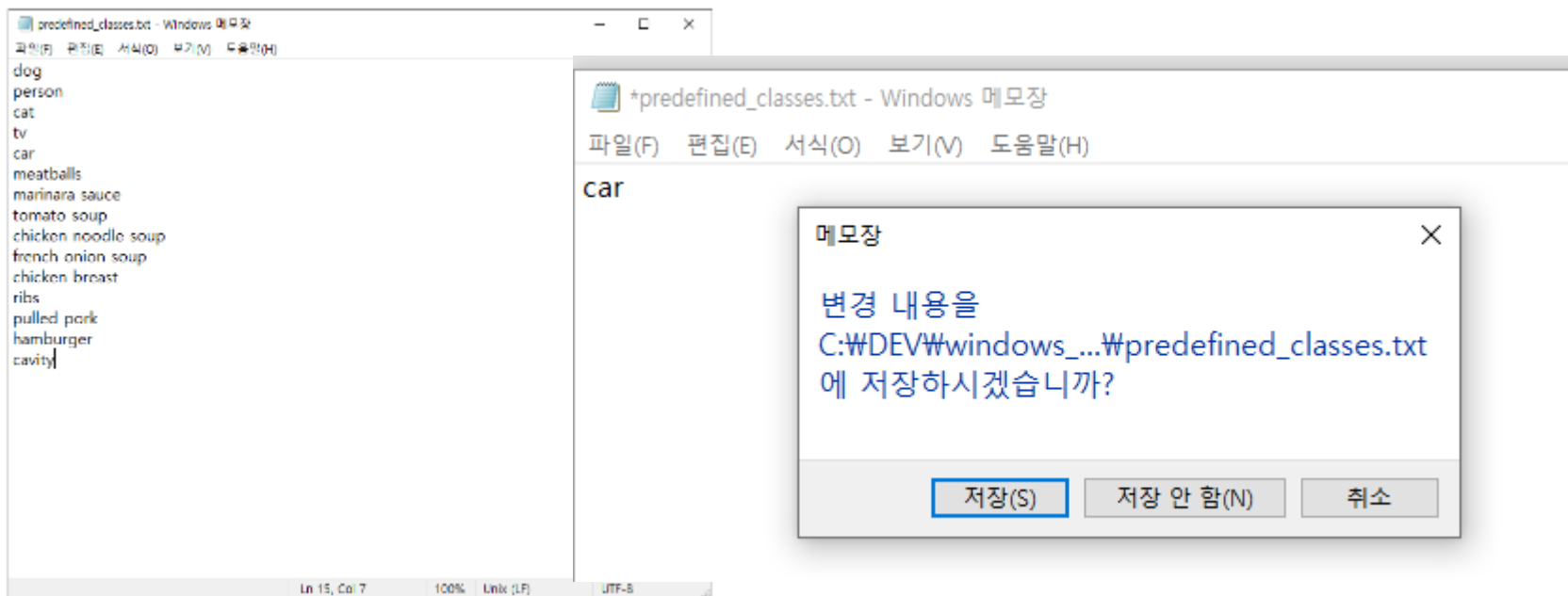
- 라벨링하고자 하는 class를 정의해주는 것과, YOLOv5의 경우 xml이 아닌 txt 파일 형식이 필요한데, labellmg의 기본값이 xml이기 때문에 이를 변경해주고 작업해야 한다.
- Class 정의



4. YOLO

Class 정의

- labellmg를 위치한 디렉토리 아래에 Data 디렉토리로 들어가면 predefined_classes.txt 라는 파일이 있다. 해당 파일을 편집기를 통해 열어주자. 텍스트 파일을 열어보면 다음과 같이 class들이 정의 되어있는데 이를 내가 원하는 class들로 변경해주면 된다.



4. YOLO

저장 방식 변경

- 라벨링을 진행할 때 저장되는 파일의 형식을 변경해줘야 한다. PascalVOC를 눌러 YOLO로 변경해주자. labellmg를 실행할 때마다 변경해주어야 한다.



4. YOLO

저장 방식 변경

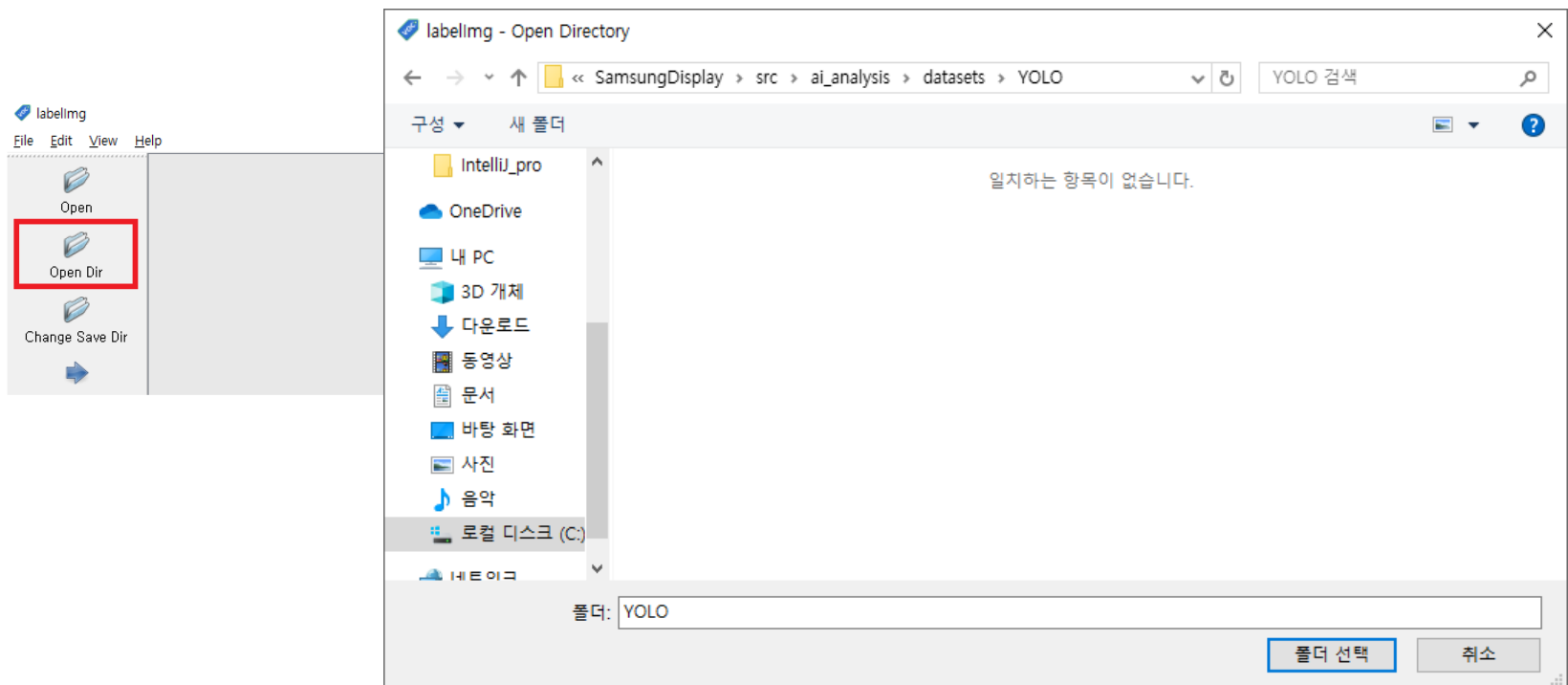
- 라벨링을 진행할 때 저장되는 파일의 형식을 변경해줘야 한다. PascalVOC를 눌러 YOLO로 변경해주자. labellmg를 실행할 때마다 변경해주어야 한다.



4. YOLO

이미지 불러오기

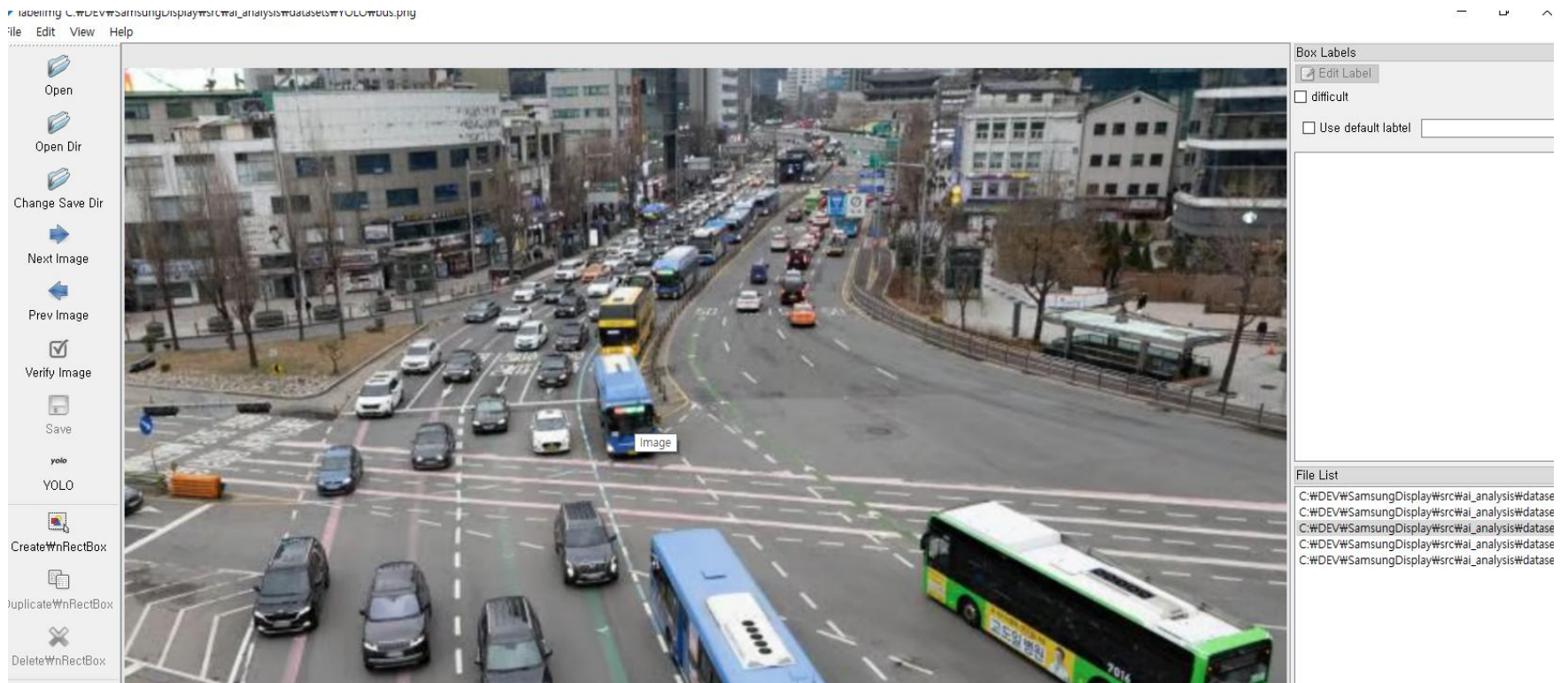
- Open Dir을 통해 이미지가 저장된 폴더를 선택해주자.



4. YOLO

이미지 불러오기

- Open Dir을 통해 이미지가 저장된 폴더를 선택해주자.



4. YOLO

단축키

- 유용한 단축키

Hotkeys

Ctrl + u	Load all of the images from a directory
Ctrl + r	Change the default annotation target dir
Ctrl + s	Save
Ctrl + d	Copy the current label and rect box
Ctrl + Shift + d	Delete the current image
Space	Flag the current image as verified
w	Create a rect box
d	Next image
a	Previous image
del	Delete the selected rect box
Ctrl++	Zoom in
Ctrl--	Zoom out
↑→↓←	Keyboard arrows to move selected rect box



4. YOLO

단축키

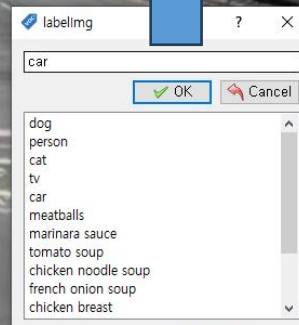
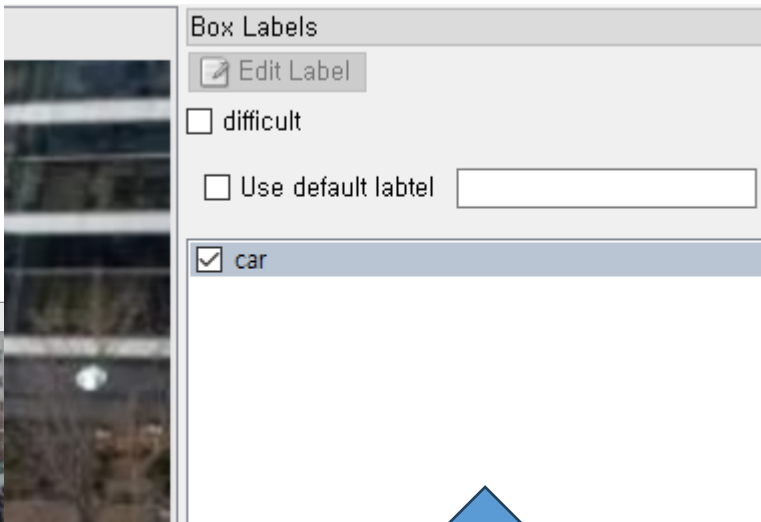
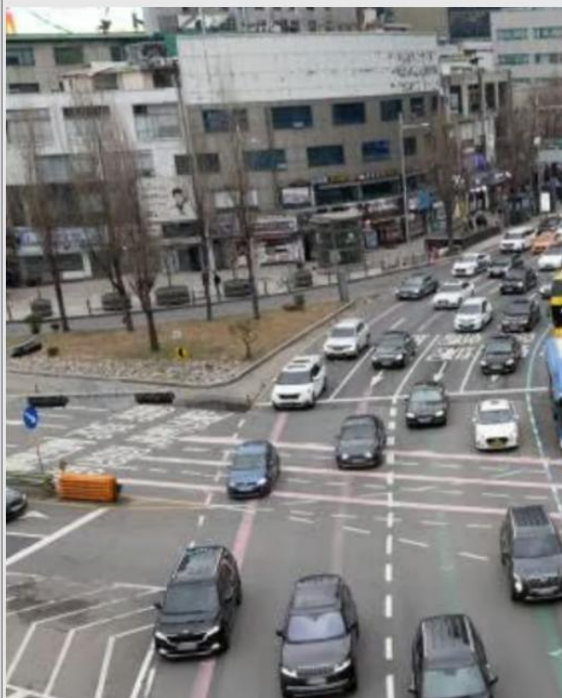
- 라벨링을 진행해보자. **W**를 누른 후 시작점부터 끝점까지 드래그 하여 Bounding Box의 영역을 설정해주자. 드래그를 종료하면 다음과 같이 class를 선택하는 창이 나오는데, 해당하는 class를 선택해주면 된다. 상단의 검색 기능을 포함하고 있으므로 영역을 지정 후 해당하는 class의 첫 글자를 입력하여 빠르게 class를 찾을 수 있다.

4. YOLO

단축키

● W

labelimg C:\DEV\SamsungDisplay\src\ai_analysis\dataset\YOLO\bus.jpg
File Edit View Help

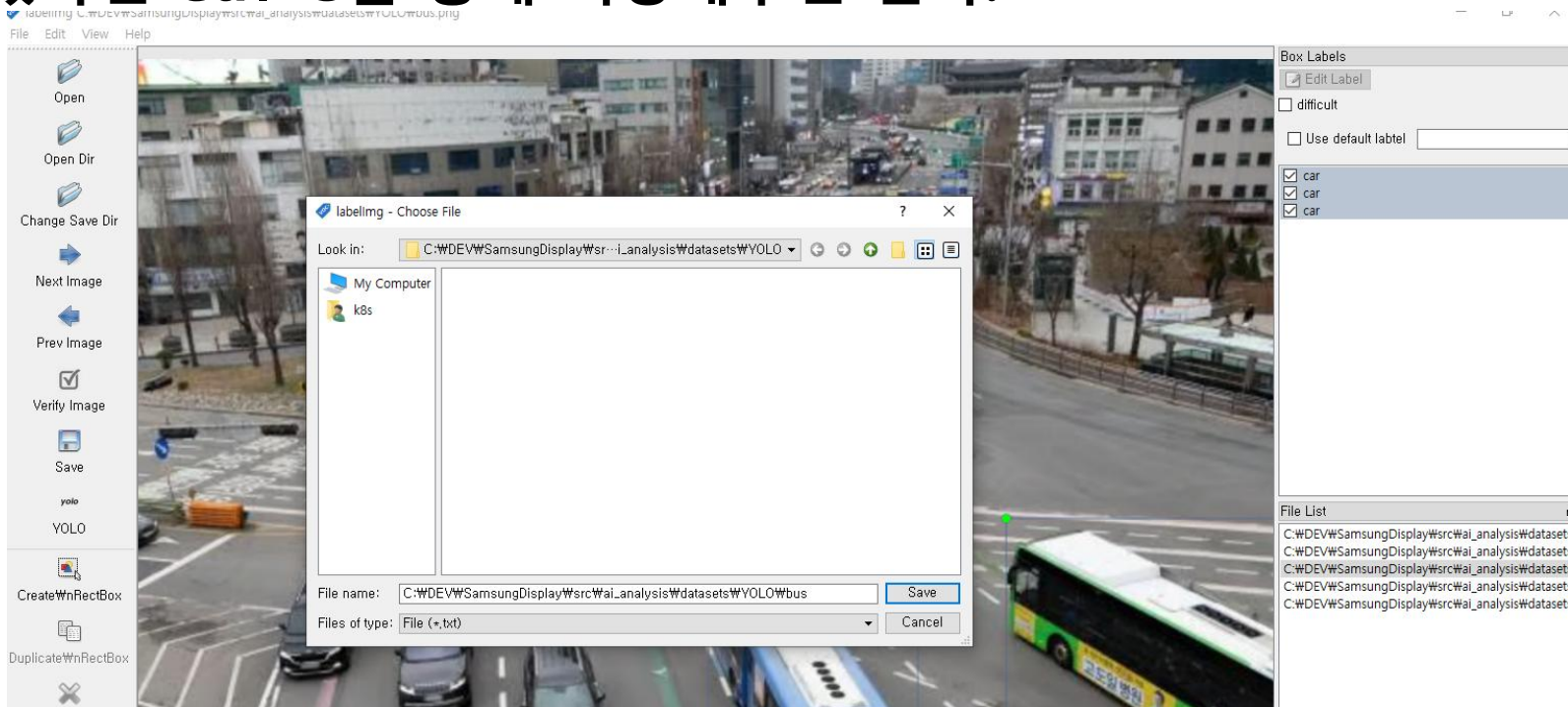


File List
C:\DEV\SamsungDisplay\src\ai_analysis\Wc
C:\DEV\SamsungDisplay\src\ai_analysis\Wc
C:\DEV\SamsungDisplay\src\ai_analysis\Wc
C:\DEV\SamsungDisplay\src\ai_analysis\Wc
C:\DEV\SamsungDisplay\src\ai_analysis\Wc

4. YOLO

단축키

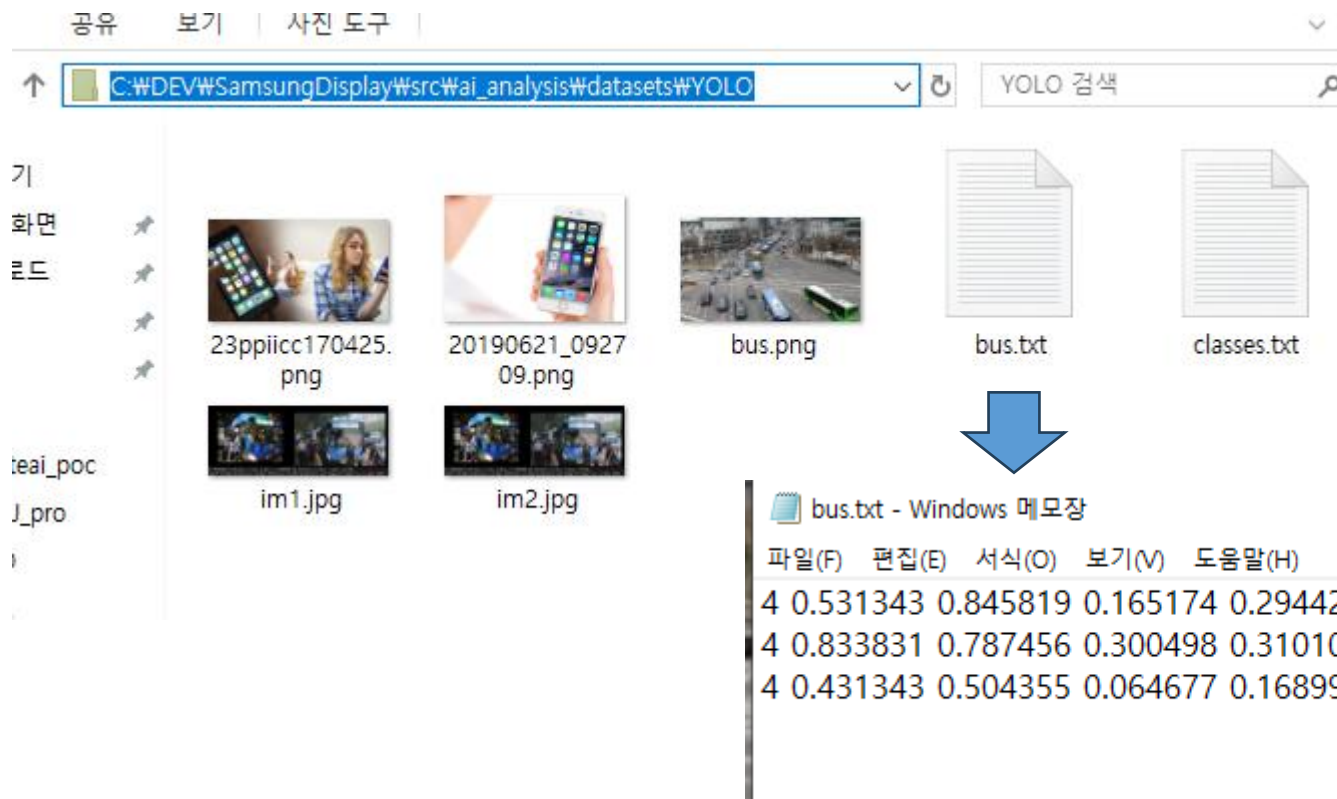
- 정상적으로 진행되었다면 이미지에 Box 영역이 표시되고 우측에 해당하는 label이 보일 것이다. 라벨링을 완료했다면 Ctrl+S를 통해 저장해주면 된다. 정상적으로 진행되었다면 이미지에 Box 영역이 표시되고 우측에 해당하는 label이 보일 것이다. 라벨링을 완료했다면 Ctrl+S를 통해 저장해주면 된다.



4. YOLO

단축키

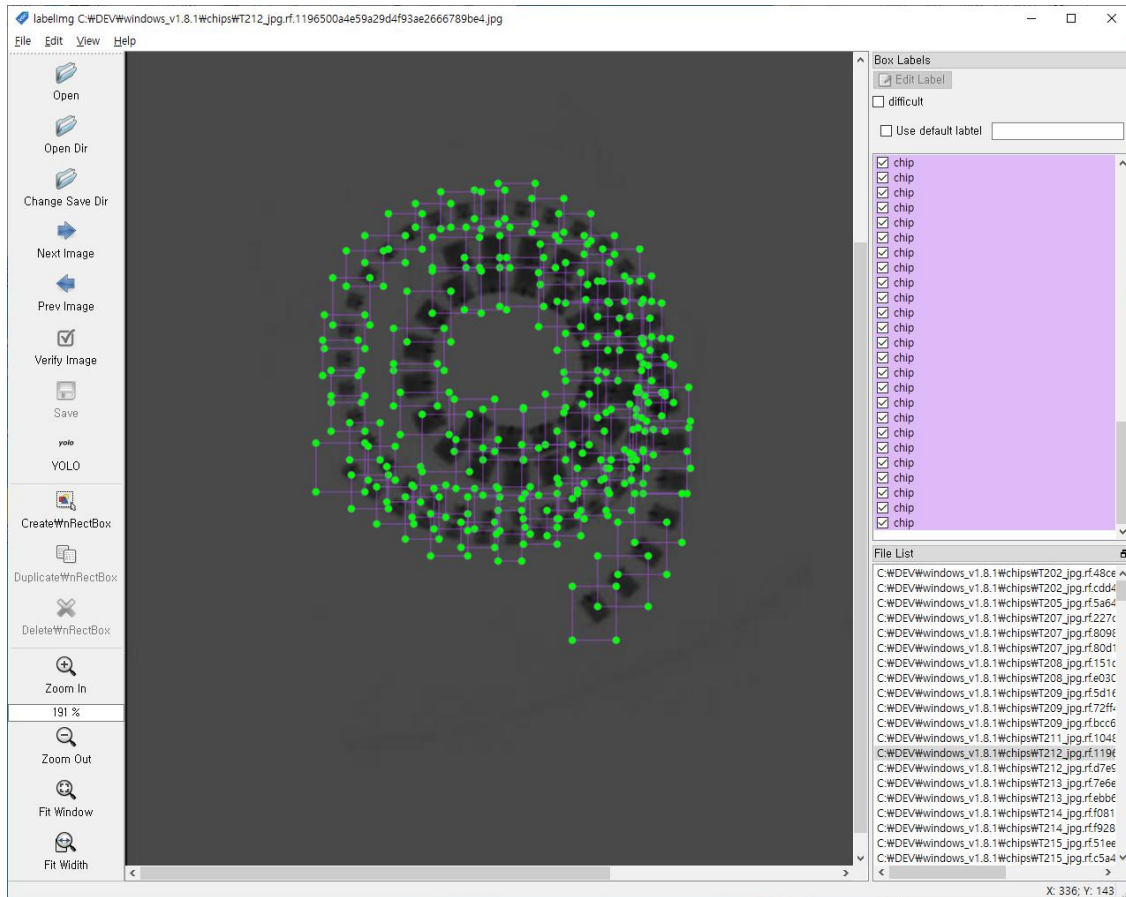
- 라벨링된 파일을 살펴보면 다음과 같이 class 번호와 좌표가 저장된 것을 확인할 수 있다.



4. YOLO

라벨 결과 화면

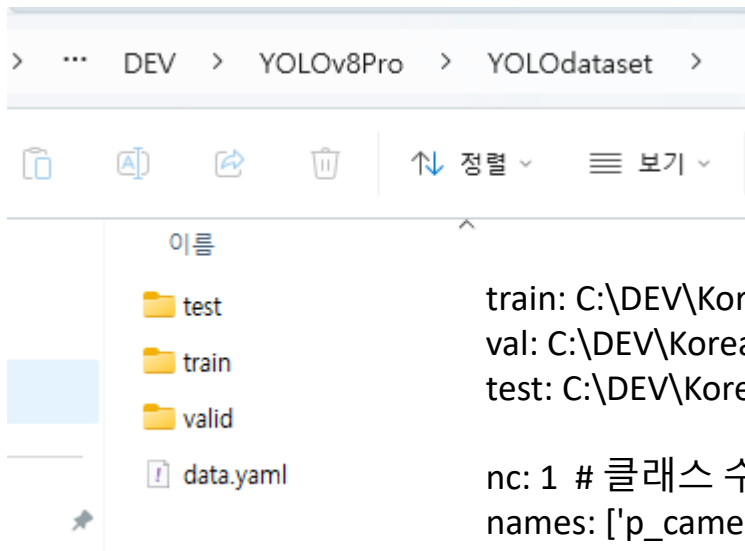
- 최대한 박스 작업을 해야 함



4. YOLO

YOLOv8 커스텀 데이터 학습하기

- YOLOv8 은 사전학습 모델종류와 데이터 위치를 나타내는 yaml 파일만 바꾸어 주면, 동일한 Python API 로 Object Detection 과 Image Segmentation 구현이 가능합니다.



4. YOLO

YOLOv8 커스텀 데이터 학습하기

data architecture

yaml 파일에서 설정한 디렉토리 위치대로 구조를 설정하고, train/valid/test에는 각각 images, labels folder를 생성한다.

각 images, labels 파일에는 image file과 label(annotation text) file을 위치하면 되며, 여기서 훈련 시 train/valid 비율은 7.5:2.5정도로 설정했다.

```
C:\DEV\KoreaTech>tree
폴더 PATH의 목록입니다.
볼륨 일련 번호는 BED0-C858입니다.
C:.
├─YOLO
│   └─windows_v1.8.1
│       └─data
└─YOLOv8Pro
    └─YOLOdataset
        ├──test
        │   ├──images
        │   └─labels
        ├──train
        │   ├──images
        │   └─labels
        └─valid
            ├──images
            └─labels

C:\DEV\KoreaTech>
```



4. YOLO

YOLOv8 커스텀 데이터 학습하기

mkdir

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\test\images

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\test\labels

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\train\images

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\train\labels

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\valid\images

C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\valid\labels

```
C:\DEV\YOLOv8Pro\YOLOdataset>tree /f
```

폴더 PATH의 목록입니다.

폴더 이름 번호는 FC5F-C327입니다.

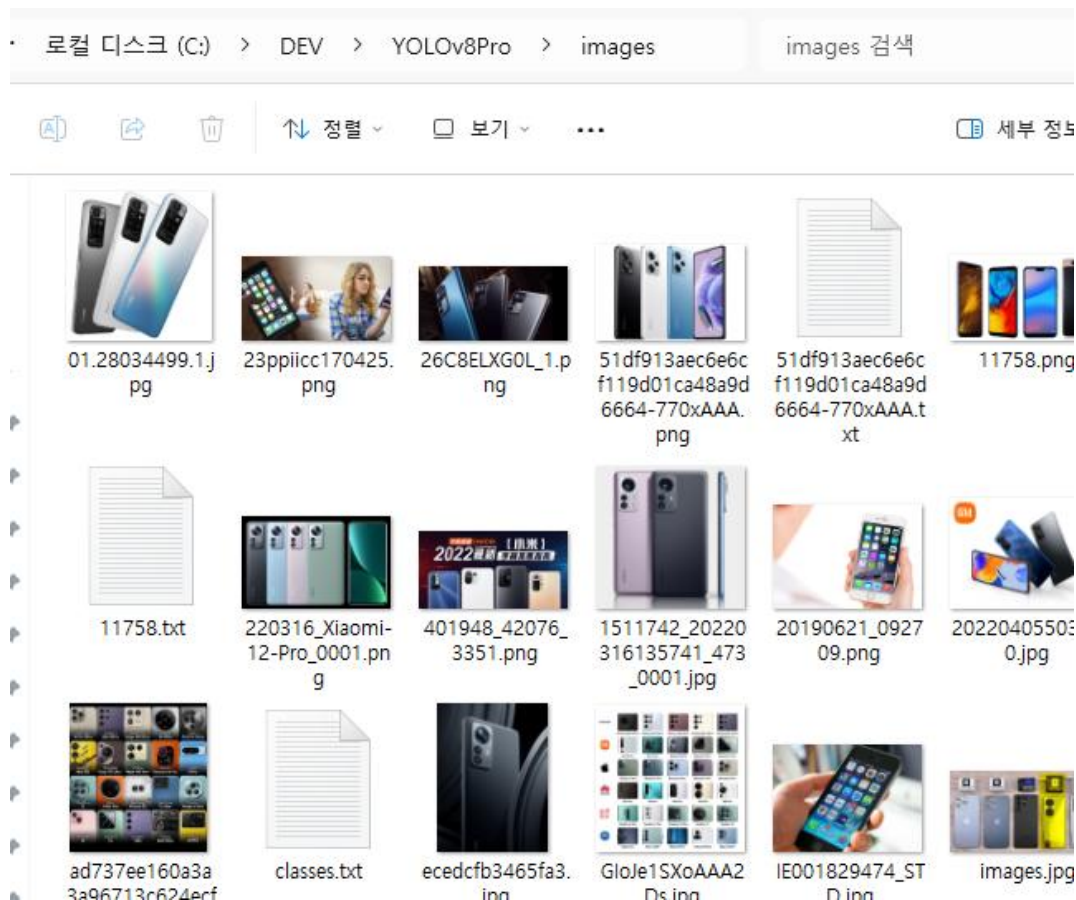
```
C:\DEV\YOLOv8Pro\YOLOdataset>tree /f
C:\DEV\YOLOv8Pro\YOLOdataset
├── data.yaml
├── test
│   ├── images
│   │   ├── 11758.png
│   │   └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│   └── labels
│       ├── 11758.txt
│       └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
├── train
│   ├── labels.cache
│   ├── images
│   │   ├── 11758.png
│   │   └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│   └── labels
│       ├── 11758.txt
│       └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
└── valid
    ├── labels.cache
    ├── images
    │   ├── 11758.png
    │   └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
    └── labels
        ├── 11758.txt
        └── 51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
```

```
C:\DEV\YOLOv8Pro\YOLOdataset>
```

4. YOLO

YOLOv8 커스텀 데이터 만들기

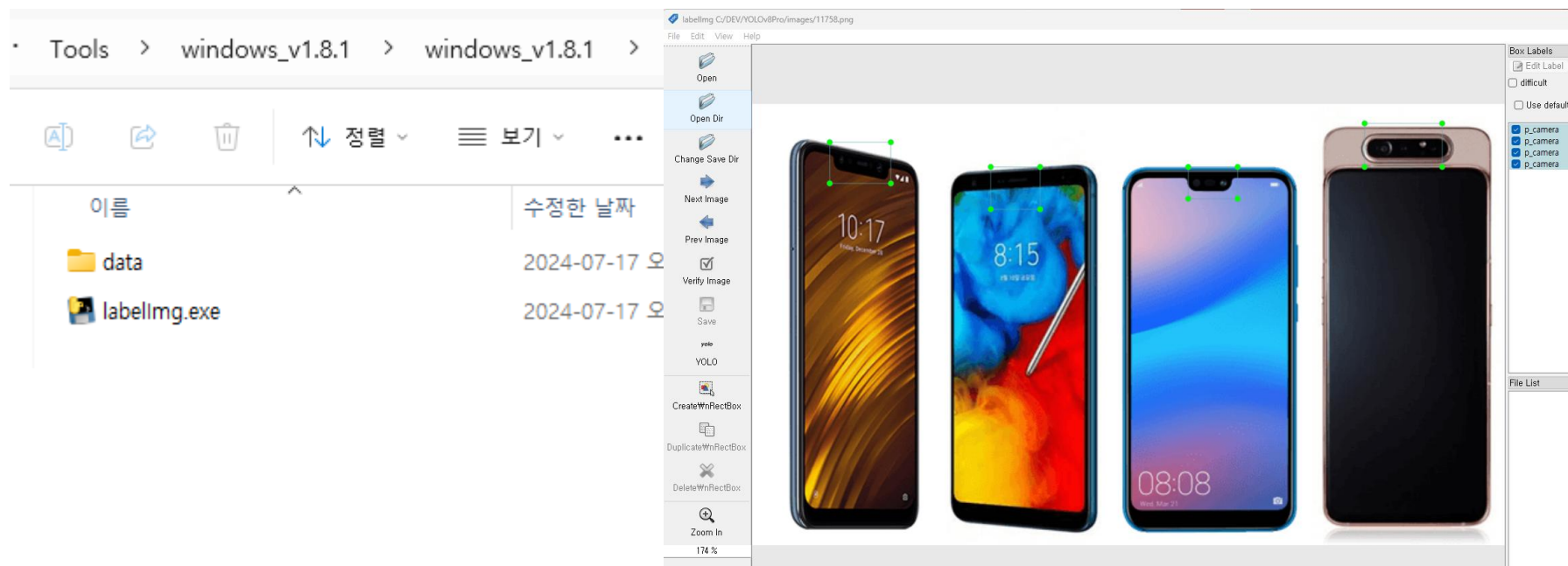
데이터를 수집합니다.



4. YOLO

YOLOv8 커스텀 데이터 만들기

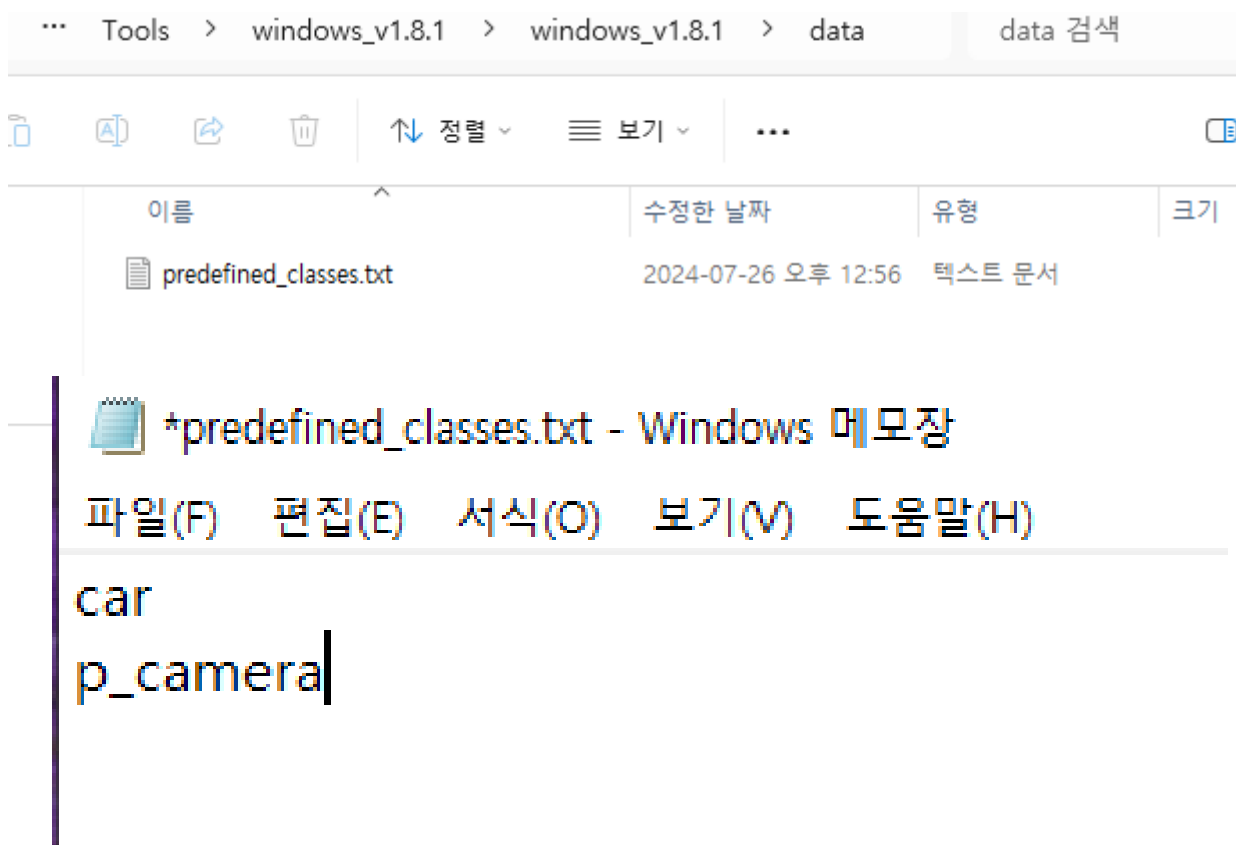
labellmg를 이용해 라벨링을 합니다.



4. YOLO

YOLOv8 커스텀 데이터 만들기

labellmg를 이용해 라벨링 시 predefined_classes.txt 파일 내용을 작업할 클래스명으로 초기화하시고 하셔야 합니다.



4. YOLO

YOLOv8 커스텀 데이터 만들기

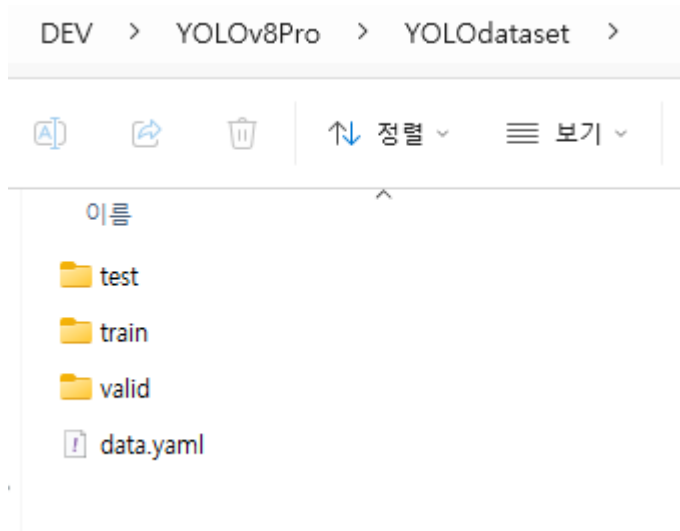
labellmg를 이용해 라벨링된 파일을 아키텍처에 맞게 배치합니다.

51df913aec6e6cf119d01ca48a9d6664-770xAAA.png

51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt

11758.png

11758.txt



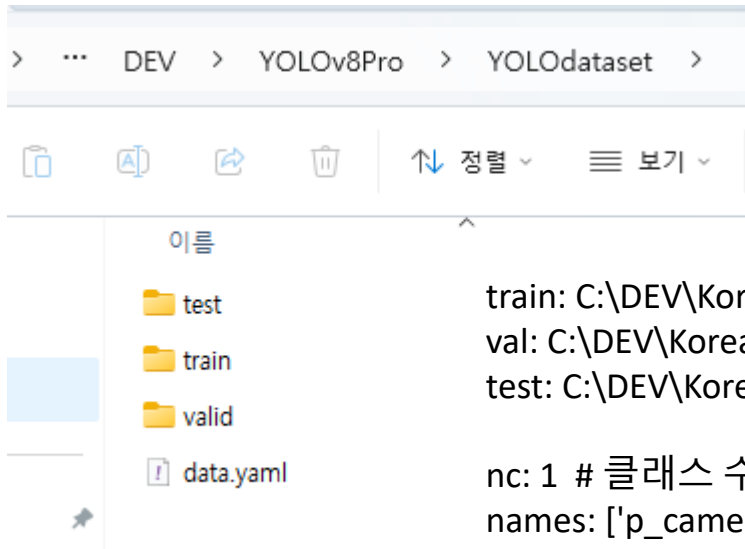
```
C:\DEV\YOLOv8Pro\YOLOdataset>tree /f
폴더 PATH의 목록입니다.
볼륨 일련 번호는 FC5F-C327입니다.
C:.
| data.yaml
|-- test
|   |-- images
|   |   11758.png
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
|   |-- labels
|   |   11758.txt
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
|-- train
|   |-- labels.cache
|   |-- images
|   |   11758.png
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
|   |-- labels
|   |   11758.txt
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
|-- valid
|   |-- labels.cache
|   |-- images
|   |   11758.png
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
|   |-- labels
|   |   11758.txt
|   |   51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
```

C:\DEV\YOLOv8Pro\YOLOdataset>

4. YOLO

YOLOv8 커스텀 데이터 학습하기

- 데이터 위치를 나타내는 yaml 파일을 바꿔 줍니다.



```
train: C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\train\images
val: C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\valid\images
test: C:\DEV\KoreaTech\YOLOv8Pro\YOLOdataset\test\images

nc: 1 # 클래스 수
names: ['p_camera'] # 클래스 이름
```

4. YOLO

YOLOv8 커스텀 데이터 학습하기

- 준비가 되면 학습을 진행합니다.

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>cd C:\DEV\YOLOv8Pro
```

```
(py3_10_yolov8) C:\DEV\YOLOv8Pro>
```

```
(py3_10_yolov8) C:\DEV\YOLOv8Pro>yolo task=detect mode=train model=yolov8s.pt data=YOLOdataset\data.yaml  
epochs=100 imgsz=640
```

Ultralytics YOLOv8.2.66 🚀 Python-3.10.11 torch-2.3.1+cpu CPU (13th Gen Intel Core(TM) i7-13700)
engine\trainer: task=detect, mode=train, model=yolov8s.pt, data=YOLOdataset\data.yaml, epochs=100, time=None, patience=100, batch=16, imgsz=640,
save=True, save_period=-1, cache=False, device=None, workers=8, project=None, name=train19, exist_ok=False, pretrained=True, optimizer=auto, verbose=True,
seed=0, deterministic=True, single_cls=False, rect=False, cos_lr=False, close_mosaic=10, resume=False, amp=True, fraction=1.0, profile=False, freeze=None,
multi_scale=False, overlap_mask=True, mask_ratio=4, dropout=0.0, val=True, split=val, save_json=False, save_hybrid=False, conf=None, iou=0.7, max_det=300,
half=False, dnn=False, plots=True, source=None, vid_stride=1, stream_buffer=False, visualize=False, augment=False, agnostic_nms=False, classes=None,
retina_masks=False, embed=None, show=False, save_frames=False, save_txt=False, save_conf=False, save_crop=False, show_labels=True, show_conf=True,
show_boxes=True, line_width=None, format=torchscript, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opset=None, workspace=4,
nms=False, lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=7.5, cls=0.5,
dfl=1.5, pose=12.0, kobj=1.0, label_smoothing=0.0, nbs=64, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0, perspective=0.0,
flipud=0.0, fliplr=0.5, bgr=0.0, mosaic=1.0, mixup=0.0, copy_paste=0.0, auto_augment=randaugment, erasing=0.4, crop_fraction=1.0, cfg=None,
tracker=botsort.yaml, save_dir=runs\detect\train19
Overriding model.yaml nc=80 with nc=3

4. YOLO

YOLOv8 커스텀 데이터 학습하기

학습 실행 결과는 C:\DEV\YOLOv8Pro\runs\detect\train\weights에 best.pt, last.pt로 저장될 것이다.

The screenshot displays a VS Code interface with a terminal window and a file explorer. The terminal window shows the output of a YOLOv8 training process, including epoch numbers, GPU memory usage, and various loss metrics. The file explorer shows the directory structure where the training results are saved, specifically the 'weights' folder containing 'best.pt' and 'last.pt' files.

Terminal Output:

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/100	0G	1.949	2.5	1.579	9	640: 100% 1/1 [00:06<00:00, 6.16s/it]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 [00:01<00:00, 1.66s/it]
	all	2	7	0.38	0.714	0.454 0.172
6/100	0G	1.483	1.788	1.242	11	640: 100% 1/1 [00:06<00:00, 6.13s/it]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 [00:01<00:00, 1.61s/it]
	all	2	7	0.745	0.839	0.797 0.368
7/100	0G	1.951	2.107	1.451	10	640: 100% 1/1 [00:06<00:00, 6.68s/it]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 [00:01<00:00, 1.75s/it]
	all	2	7	0.974	0.714	0.827 0.41
8/100	0G					

File Explorer:

이름	수정한 날짜
best.pt	2024-07-26 오후
last.pt	2024-07-26 오후



4. YOLO

YOLOv8 커스텀 데이터 학습 모델 사용하기

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('C:\\DEV\\YOLOv8Pro\\runs\\detect\\train\\weights\\best.pt') # pretrained YOLOv8n model

model.predict(source="0", show=True, stream=True, classes=0) # [0, 3, 5] for multiple classes

# Define image file path
image_folder = 'C:\\DEV\\SamsungDisplay\\src\\ai_analysis\\datasets\\YOLO\\'

# Define image file names
image_file_1 = '23ppiicc170425.png'
image_file_2 = 'bus.png'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2]) # return a list of Results objects
model.predict(source="0", show=True, stream=True, classes=0)

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs

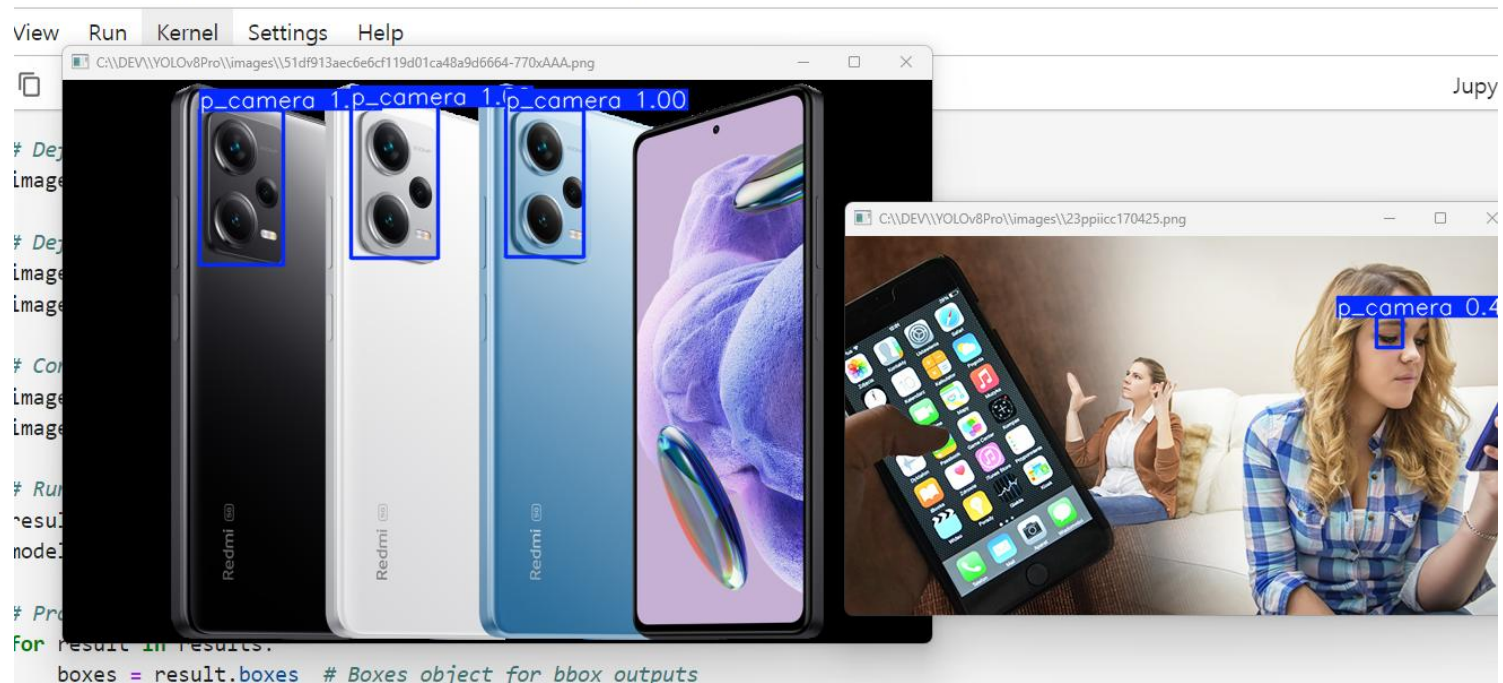
    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

4. YOLO

YOLOv8 커스텀 데이터 학습 모델 사용하기

결과





4. YOLO

YOLOv8 커스텀 데이터 학습 모델 사용하기

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('C:\\DEV\\YOLOv8Pro\\runs\\detect\\train\\weights\\best.pt') # pretrained YOLOv8n model

model.predict(source="0", show=True, stream=True, classes=0) # [0, 3, 5] for multiple classes

# Define image file path
image_folder = 'C:\\DEV\\SamsungDisplay\\src\\ai_analysis\\datasets\\YOLO\\'

# Define image file names
image_file_1 = '23ppiicc170425.png'
image_file_2 = 'bus.png'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2]) # return a list of Results objects
model.predict(source="0", show=True, stream=True, classes=0)

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bbox outputs
    masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!